Retrospective Evaluation of Soccer Players (REvSoc)

Seshadhri S

2025-07-18

2022-23 season

Goalkeeper Rating System: Methodology & Validation

In this analysis, I developed a multidimensional goalkeeper rating model designed to evaluate both conventional and modern aspects of goalkeeping performance. The model incorporates traditional shot-stopping metrics alongside advanced measures of distribution, sweeping actions, and cross management to reflect the evolving demands of the position.

Method Overview

The rating was divided into two core components:

Conventional Keeper Rating (CKR) — incorporating metrics like Saves, Goals Against (GA), Save Percentage, and Post-Shot Expected Goals (PSxG) overperformance.

Sweeper Keeper Rating (SKR) — capturing distribution accuracy, launch rates, defensive actions outside the box, average distance of defensive actions, and cross-stopping percentages.

Both components were normalized relative to their averages and combined to produce a Total Goalkeeper Rating.

Validation Process

To ensure the model was both credible and informative, I validated the Total Goalkeeper Rating against three independent metrics:

1. Goals Against (GA)
2. PSxG-GA per 90 (shot-stopping over expected)
3. Save Percentage

```r
library(worldfootballR)
library(purrr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(Boruta)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v readr     2.1.5

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(FactoMineR)
library(factoextra)
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
library(ggrepel)
```

Load in FBRef Data for the Big 5 Leagues

```
big5_player_standard <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "standard", tean
big5_player_shooting <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "shooting", tean
big5_player_passing <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "passing", team_
big5_player_passing_types <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "passing_ty
big5_player_gca <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "gca", team_or_player
big5_player_defense <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "defense", team_
big5_player_possession <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "possession",
big5_player_playing_time <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "playing_tin
big5_player_misc <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "misc", team_or_play
big5_player_keepers <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "keepers", team_
big5_player_keepers_adv <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "keepers_adv

prem_league_table <- fb_season_team_stats(country = "ENG", gender = "M", season_end_year = "2023", tier
seriea_league_table <- fb_season_team_stats(country = "ITA", gender = "M", season_end_year = "2023", tie
ligue1_league_table <- fb_season_team_stats(country = "FRA", gender = "M", season_end_year = "2023", tie
laliga_league_table <- fb_season_team_stats(country = "ESP", gender = "M", season_end_year = "2023", tie
bundesliga_league_table <- fb_season_team_stats(country = "GER", gender = "M", season_end_year = "2023"
```

Goalkeeper Ratings

```r
##------Goalkeeper----
#>Have to divide this into Conventional Keeper Rating (CKR) and Sweeper Keeper Rating (SKR)
#>Take the keepers away from the playing times dataframe

gk_playing_time <- big5_player_playing_time %>% filter(Pos == "GK") %>% filter(MP_Playing.Time != 0)

df_list <- list(gk_playing_time, big5_player_keepers, big5_player_keepers_adv)
common_columns <- Reduce(intersect, lapply(df_list, colnames))

merged_df <- inner_join(big5_player_keepers, big5_player_keepers_adv, gk_playing_time, by = c("Season_E

gk_above_900 <- merged_df %>% filter(Min_Playing >= 900)

keepers_numeric <-gk_above_900 %>% select(where(is.numeric))

# Optional: remove NAs
keepers_numeric <- keepers_numeric[, -1]
keepers_numeric <- na.omit(keepers_numeric)

# Scale the data (important for PCA since units differ)
keepers_scaled <- scale(keepers_numeric)
keepers_scaled <- keepers_scaled[ ,-1]
# Run PCA
keepers_pca <- PCA(keepers_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
keepers_pca$eig
```

```
##           eigenvalue percentage of variance cumulative percentage of variance
## comp 1  1.709957e+01           3.799903e+01                          37.99903
## comp 2  6.889010e+00           1.530891e+01                          53.30795
## comp 3  3.819156e+00           8.487014e+00                          61.79496
## comp 4  3.033827e+00           6.741837e+00                          68.53680
## comp 5  2.277373e+00           5.060829e+00                          73.59763
## comp 6  1.782952e+00           3.962115e+00                          77.55974
## comp 7  1.636681e+00           3.637069e+00                          81.19681
## comp 8  1.324431e+00           2.943179e+00                          84.13999
## comp 9  1.052435e+00           2.338744e+00                          86.47873
## comp 10 9.961785e-01           2.213730e+00                          88.69246
## comp 11 8.046943e-01           1.788210e+00                          90.48067
## comp 12 6.884262e-01           1.529836e+00                          92.01051
## comp 13 5.512759e-01           1.225058e+00                          93.23557
## comp 14 4.830598e-01           1.073466e+00                          94.30903
## comp 15 4.476523e-01           9.947828e-01                          95.30381
## comp 16 3.848105e-01           8.551344e-01                          96.15895
## comp 17 3.186478e-01           7.081062e-01                          96.86706
## comp 18 3.004256e-01           6.676125e-01                          97.53467
## comp 19 2.204826e-01           4.899613e-01                          98.02463
## comp 20 1.841858e-01           4.093019e-01                          98.43393
## comp 21 1.341693e-01           2.981540e-01                          98.73208
## comp 22 1.142680e-01           2.539288e-01                          98.98601
```

```
## comp 23 1.068636e-01          2.374747e-01                          99.22349
## comp 24 9.529081e-02          2.117573e-01                          99.43525
## comp 25 7.609073e-02          1.690905e-01                          99.60434
## comp 26 3.760517e-02          8.356705e-02                          99.68790
## comp 27 3.139861e-02          6.977469e-02                          99.75768
## comp 28 2.756037e-02          6.124526e-02                          99.81892
## comp 29 2.174391e-02          4.831981e-02                          99.86724
## comp 30 1.369638e-02          3.043640e-02                          99.89768
## comp 31 1.335589e-02          2.967975e-02                          99.92736
## comp 32 1.159565e-02          2.576811e-02                          99.95313
## comp 33 8.121063e-03          1.804681e-02                          99.97117
## comp 34 6.959149e-03          1.546478e-02                          99.98664
## comp 35 4.035880e-03          8.968621e-03                          99.99561
## comp 36 1.558634e-03          3.463631e-03                          99.99907
## comp 37 4.147810e-04          9.217356e-04                          99.99999
## comp 38 2.373329e-06          5.274064e-06                         100.00000
## comp 39 8.767401e-07          1.948311e-06                         100.00000
## comp 40 2.403136e-31          5.340302e-31                         100.00000
## comp 41 1.428847e-31          3.175215e-31                         100.00000
## comp 42 1.428847e-31          3.175215e-31                         100.00000
## comp 43 1.428847e-31          3.175215e-31                         100.00000
## comp 44 1.428847e-31          3.175215e-31                         100.00000
## comp 45 1.428847e-31          3.175215e-31                         100.00000
```

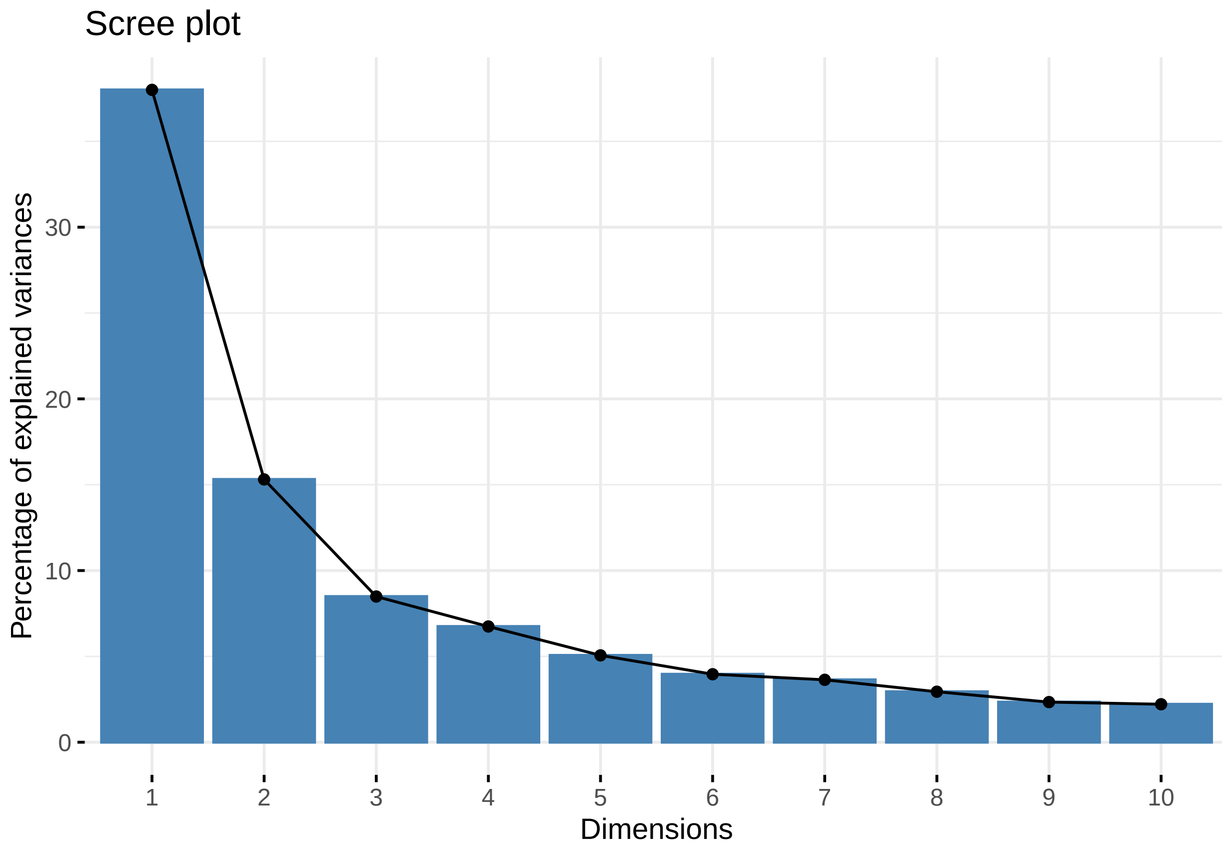
```r
# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- keepers_pca$eig[1, "percentage of variance"]
dim2_variance <- keepers_pca$eig[2, "percentage of variance"]

# Normalize so they sum to 1 (as relative weights)
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

```
## [1] "Dim.1 weight: 0.713"
```

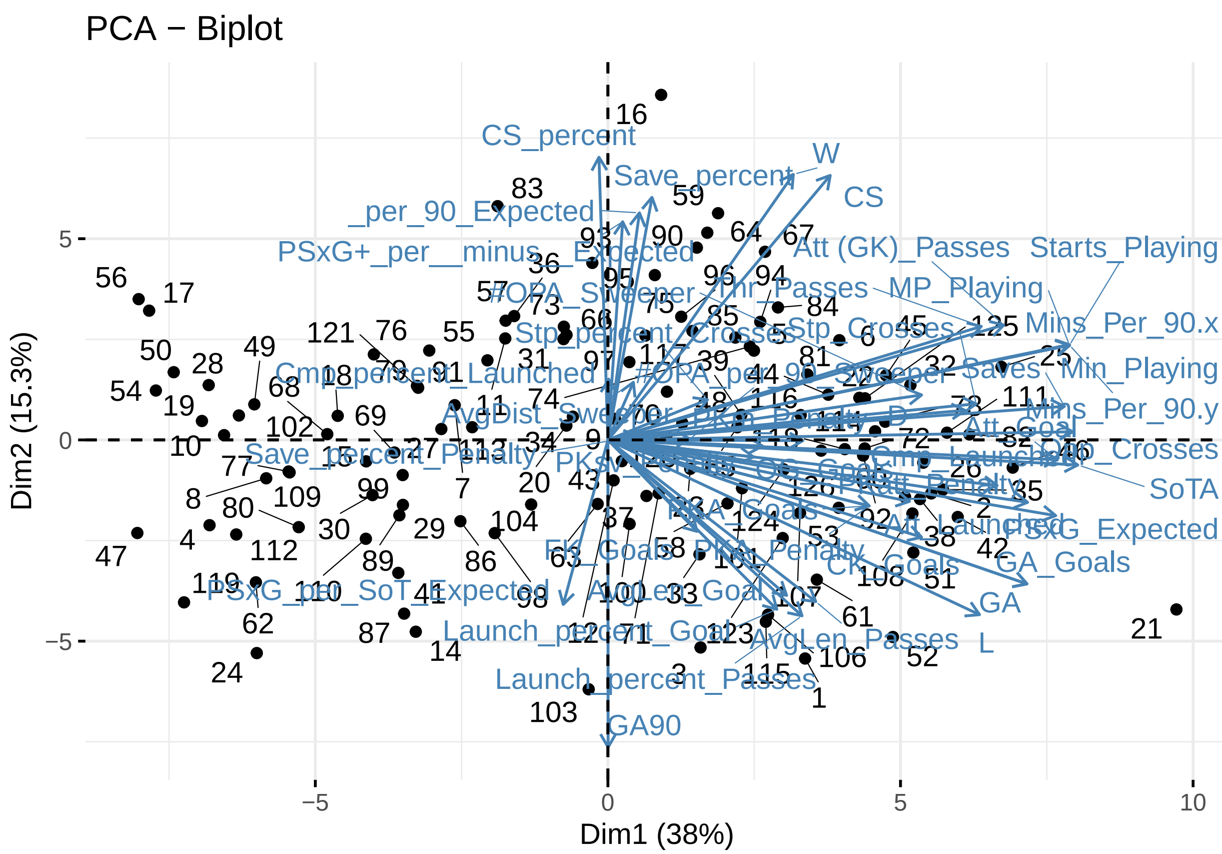
```r
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

```
## [1] "Dim.2 weight: 0.287"
```

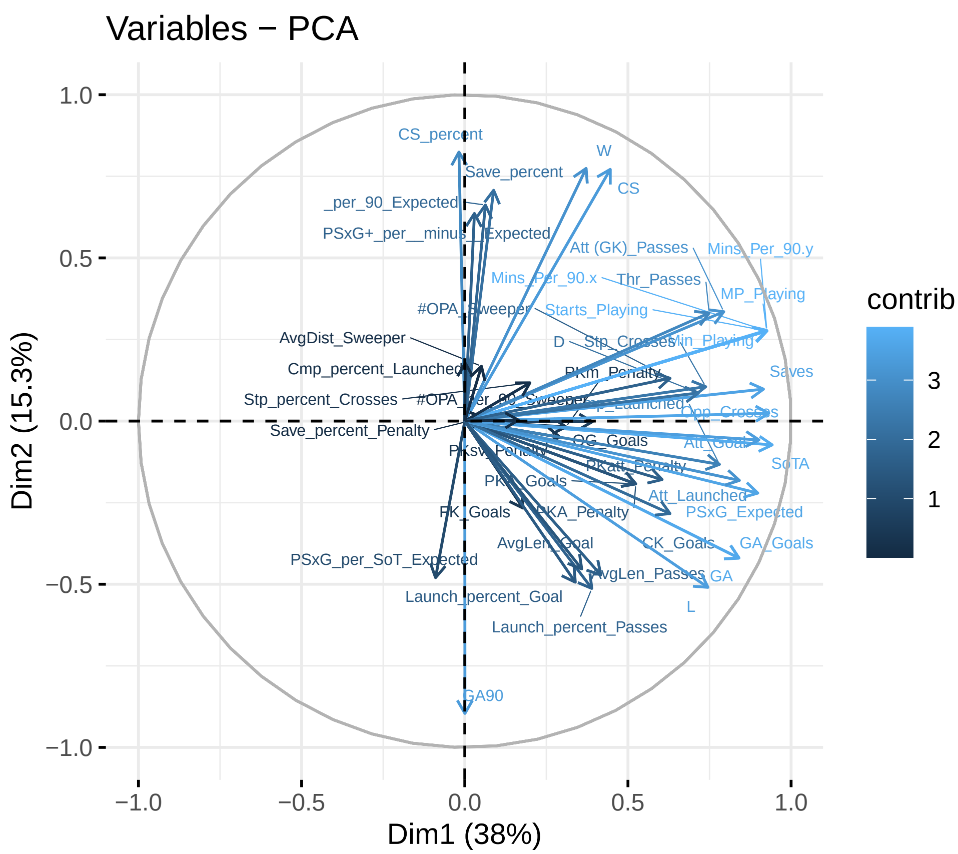
```r
# Plot the variance explained
fviz_eig(keepers_pca)
```




```
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(keepers_pca, repel = TRUE)
```



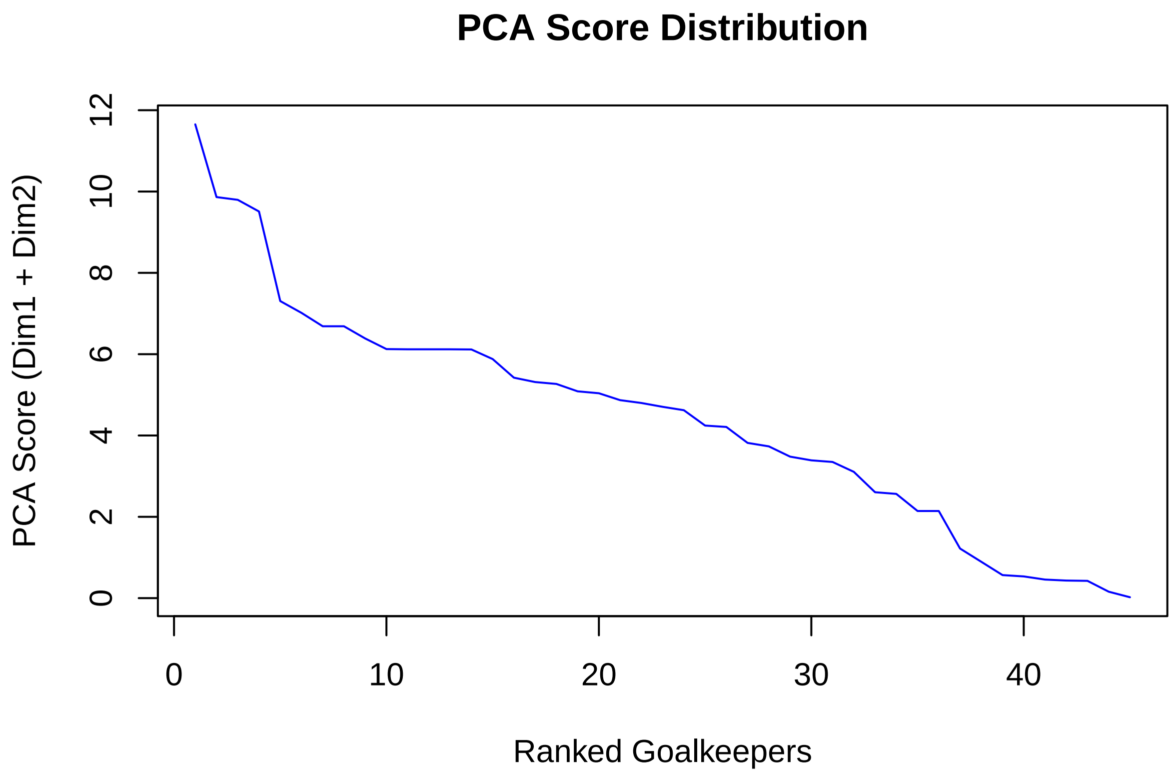

```r
# Contributions of variables to PC1 and PC2
fviz_pca_var(keepers_pca,
             col.var = "contrib",
             labelsize = 2, repel = TRUE)
```




```r
variable_contrib <- data.frame(keepers_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 6.119688
```

```r
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
     ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```



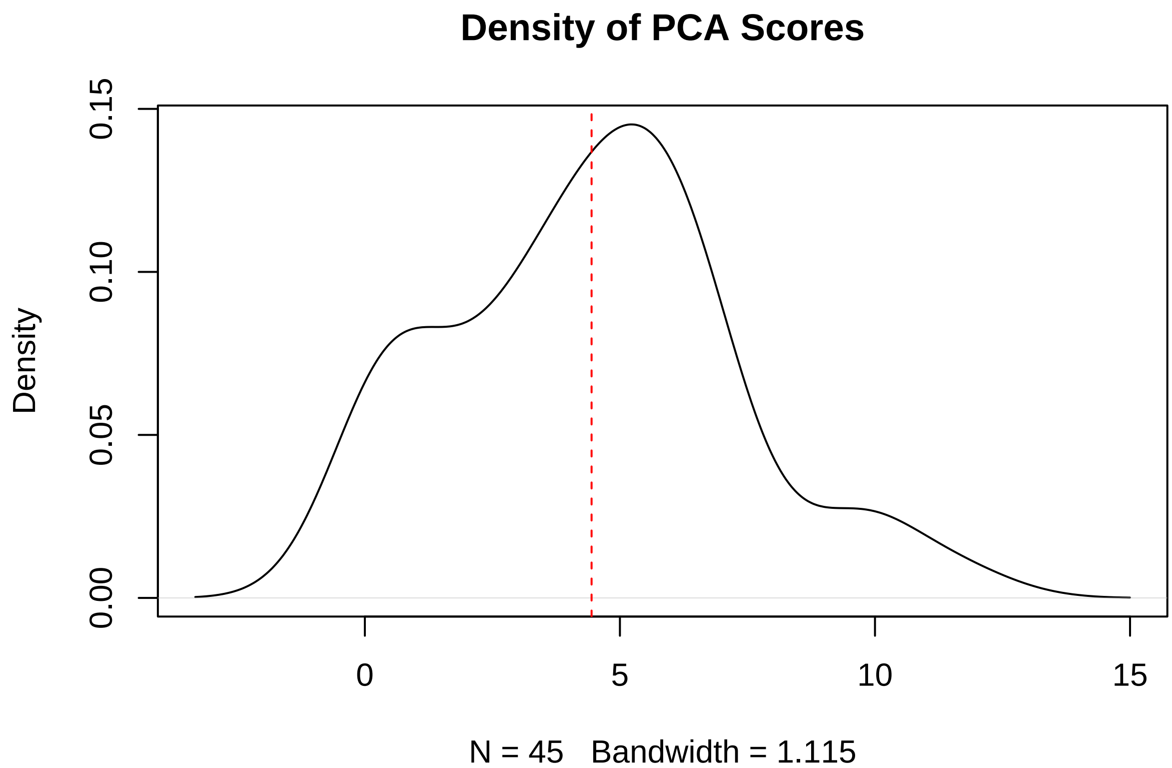

```r
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##     j1 j2 chi
## ESE 10 44  27
## EDE  7  1 NaN
```

```r
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 7.25211
```

```r
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```



The PCA variable contribution plot shows a clear differentiation between shot stopping and sweeper keeper metrics

1. OPA__per__90__Sweeper and AvgDist__Sweeper are almost overlapping and long - These two are very closely related and both contribute heavily to PC1. TL;DR: your sweeper keeper stats move together and drive a big chunk of the differences between keepers.
2. PSxG+__per___minus___Expected (the expected goals prevented stat) points in a different direction and is quite long too - Strongly influential but not correlated with the sweeper actions. It's carrying weight independently, probably defining a classic shot-stopper profile.
3. Cmp__percent__Launched and Stp__percent__Crosses have shorter vectors, contributing less to the variance on these first two components. Could mean they're either:

- Less variable between your keepers
- Or important on other PCs (PC3, PC4…)

```
CKR <- with(gk_above_900,
  (Min_Playing *
   Saves * variable_contrib["Saves", "Weight"] *
   PSxG_Expected * variable_contrib["PSxG_Expected", "Weight"]) /
  (GA * variable_contrib["GA", "Weight"] * GA * SoTA * variable_contrib["SoTA", "Weight"])
)

SKR <- with(gk_above_900,
```

```r
  (
    (Cmp_Launched/Att_Launched) * variable_contrib["Cmp_Launched", "Weight"] +
    (Stp_Crosses/Opp_Crosses) * variable_contrib["Stp_Crosses", "Weight"] +
    (AvgLen_Passes/`Att (GK)_Passes`) * variable_contrib["AvgLen_Passes", "Weight"] +
    (AvgDist_Sweeper/`#OPA_Sweeper`) * variable_contrib["AvgDist_Sweeper", "Weight"] +
    (AvgLen_Goal/Att_Goal) * variable_contrib["AvgLen_Goal", "Weight"]
  ) / Min_Playing
)

avg_CKR <- mean(CKR, na.rm = TRUE)
avg_SKR <- mean(SKR, na.rm = TRUE)

CKR_Avg <- CKR / avg_CKR
SKR_Avg <- SKR / avg_SKR

Total_GK_Rating <- (CKR_Avg * dim1_weight) + (SKR_Avg * dim2_weight)

gk_ratings_final <- gk_above_900 %>%
  mutate(
    CKR_Avg = CKR_Avg,
    SKR_Avg = SKR_Avg,
    Total_GK_Rating = Total_GK_Rating
  ) %>%
  arrange(desc(Total_GK_Rating))

gk_ratings_24starts <- gk_ratings_final %>% filter(Starts_Playing >=25) %>% select(Squad, Comp, Player,
print(head(gk_ratings_24starts))
```

```
##             Squad       Comp                Player  CKR_Avg   SKR_Avg
## 1       Barcelona    La Liga Marc-André ter Stegen 4.193117 0.5482788
## 2            Lens    Ligue 1           Brice Samba 2.181801 0.5257366
## 3 Atlético Madrid    La Liga             Jan Oblak 2.060549 0.6814057
## 4    Union Berlin Bundesliga       Frederik Rønnow 1.970255 0.7725723
## 5          Napoli    Serie A            Alex Meret 1.947903 0.4827229
## 6           Lazio    Serie A        Ivan Provedel 1.862407 0.4697271
##   Total_GK_Rating
## 1        3.146397
## 2        1.706215
## 3        1.664489
## 4        1.626306
## 5        1.527135
## 6        1.462459
```

```r
#Correlation work between GA and GKR

cor(gk_ratings_final$Total_GK_Rating, gk_ratings_final$GA, use = "complete.obs", method = "spearman")
```

```
## [1] -0.8337625
```

```r
cor(gk_ratings_final$Total_GK_Rating, gk_ratings_final$`PSxG+_per__minus__Expected`, use = "complete.obs
```

```
## [1] 0.6251774
```

```r
cor(gk_ratings_final$Total_GK_Rating, (gk_ratings_final$PSxG_Expected - gk_ratings_final$GA) / (gk_ratin
```

```
## [1] 0.6052242
```

```r
cor(gk_ratings_final$Total_GK_Rating, gk_ratings_final$Save_percent, use = "complete.obs", method = "spe
```

```
## [1] 0.6415647
```

```r
gk_ratings_league_avg <- gk_ratings_final %>% group_by(Comp) %>% summarise(Avg_GKR = mean(Total_GK_Ratin
print(gk_ratings_league_avg)
```

```
## # A tibble: 5 x 2
##   Comp           Avg_GKR
##   <chr>            <dbl>
## 1 Bundesliga       0.922
## 2 La Liga          1.21
## 3 Ligue 1          0.981
## 4 Premier League   0.833
## 5 Serie A          1.02
```

```r
gk_psxg_league_avg <- gk_ratings_final %>% group_by(Comp) %>% summarise(Avg_PSxG = mean(PSxG_Expected))
print(gk_psxg_league_avg)
```

```
## # A tibble: 5 x 2
##   Comp           Avg_PSxG
##   <chr>             <dbl>
## 1 Bundesliga         35.6
## 2 La Liga            30.5
## 3 Ligue 1            39.3
## 4 Premier League     36.6
## 5 Serie A            32.3
```

```r
cor(gk_ratings_league_avg$Avg_GKR, gk_psxg_league_avg$Avg_PSxG, method = "spearman")
```

```
## [1] -0.7
```

```r
#Split them into Each League

pl_gk_ratings <- gk_ratings_final %>% filter(Comp == "Premier League")
bundesliga_gk_ratings <- gk_ratings_final %>% filter(Comp == "Bundesliga")
seriea_gk_ratings <- gk_ratings_final %>% filter(Comp == "Serie A")
ligue1_gk_ratings <- gk_ratings_final %>% filter(Comp == "Ligue 1")
laliga_gk_ratings <- gk_ratings_final %>% filter(Comp == "La Liga")

#Awards for each league
pl_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Premier League")
bundesliga_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Bundesliga")
seriea_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Serie A")
ligue1_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "Ligue 1")
laliga_gk_ratings_awards <- gk_ratings_24starts %>% filter(Comp == "La Liga")
```

Interpretation

These correlations confirm that the model is strongly aligned with key outcome metrics while maintaining balance:

High correlation with Goals Against indicates outcome relevance.

Moderate correlations with PSxG-GA per 90 and Save Percentage confirm the model's shot-stopping validity without overfitting to a single dimension.

The moderate strength of these relationships reflects the rating's inclusion of modern, non-shot-stopping actions, enhancing its relevance to contemporary goalkeeper profiles.

Conclusion

This multidimensional rating offers a more comprehensive view of goalkeeper performance than conventional single-metric models. It captures the hybrid nature of the modern goalkeeper — one expected to contribute both in traditional shot-stopping and proactive defensive play — and stands up well to statistical validation against key performance outcomes.

Outfield Player Ratings

Defensive Player Ratings

```r
#>DEFENSIVE PLAYER RATING

outfield_playing_time <- big5_player_playing_time %>% filter(Pos != "GK") %>% filter(Min_Playing.Time >=
outfield_df_list <- list(outfield_playing_time, big5_player_defense, big5_player_gca, big5_player_misc,
                         big5_player_passing, big5_player_passing_types, big5_player_possession,
                         big5_player_shooting, big5_player_standard)
common_columns_outfield <- Reduce(intersect, lapply(outfield_df_list, colnames))
print(common_columns_outfield)
```

```
## [1] "Season_End_Year" "Squad"           "Comp"            "Player"
## [5] "Nation"          "Pos"             "Age"             "Born"
## [9] "Url"
```

```r
outfield_data <- outfield_df_list %>%
  reduce(left_join, by = c("Season_End_Year", "Squad", "Comp", "Player", "Nation", "Pos", "Age", "Born"
outfield_data <- outfield_data %>%
  select(-Url)

outfield_defensive_data <- outfield_data %>% select(Season_End_Year, Squad, Comp, Player, Nation, Pos, A

outfield_above_900 <- outfield_defensive_data %>% filter(Min_Playing.Time >= 900)

outfield_numeric <-outfield_above_900 %>% select(where(is.numeric))

# Optional: remove NAs
outfield_numeric <- outfield_numeric[, -1]
outfield_numeric <- na.omit(outfield_numeric)
```

```r
# Scale the data (important for PCA since units differ)
outfield_scaled <- scale(outfield_numeric)
outfield_scaled <- outfield_scaled[ ,-1]
# Run PCA
outfield_pca <- PCA(outfield_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
outfield_pca$eig
```

```
##         eigenvalue percentage of variance cumulative percentage of variance
## comp 1  6.71472293             44.7648195                          44.76482
## comp 2  2.08801068             13.9200712                          58.68489
## comp 3  1.75098914             11.6732609                          70.35815
## comp 4  1.07521413              7.1680942                          77.52625
## comp 5  0.78492839              5.2328559                          82.75910
## comp 6  0.66544208              4.4362805                          87.19538
## comp 7  0.51681158              3.4454105                          90.64079
## comp 8  0.31769273              2.1179516                          92.75874
## comp 9  0.29553701              1.9702467                          94.72899
## comp 10 0.24045682              1.6030454                          96.33204
## comp 11 0.18401766              1.2267844                          97.55882
## comp 12 0.14365660              0.9577107                          98.51653
## comp 13 0.13317800              0.8878533                          99.40439
## comp 14 0.06216283              0.4144189                          99.81880
## comp 15 0.02717942              0.1811961                         100.00000
```

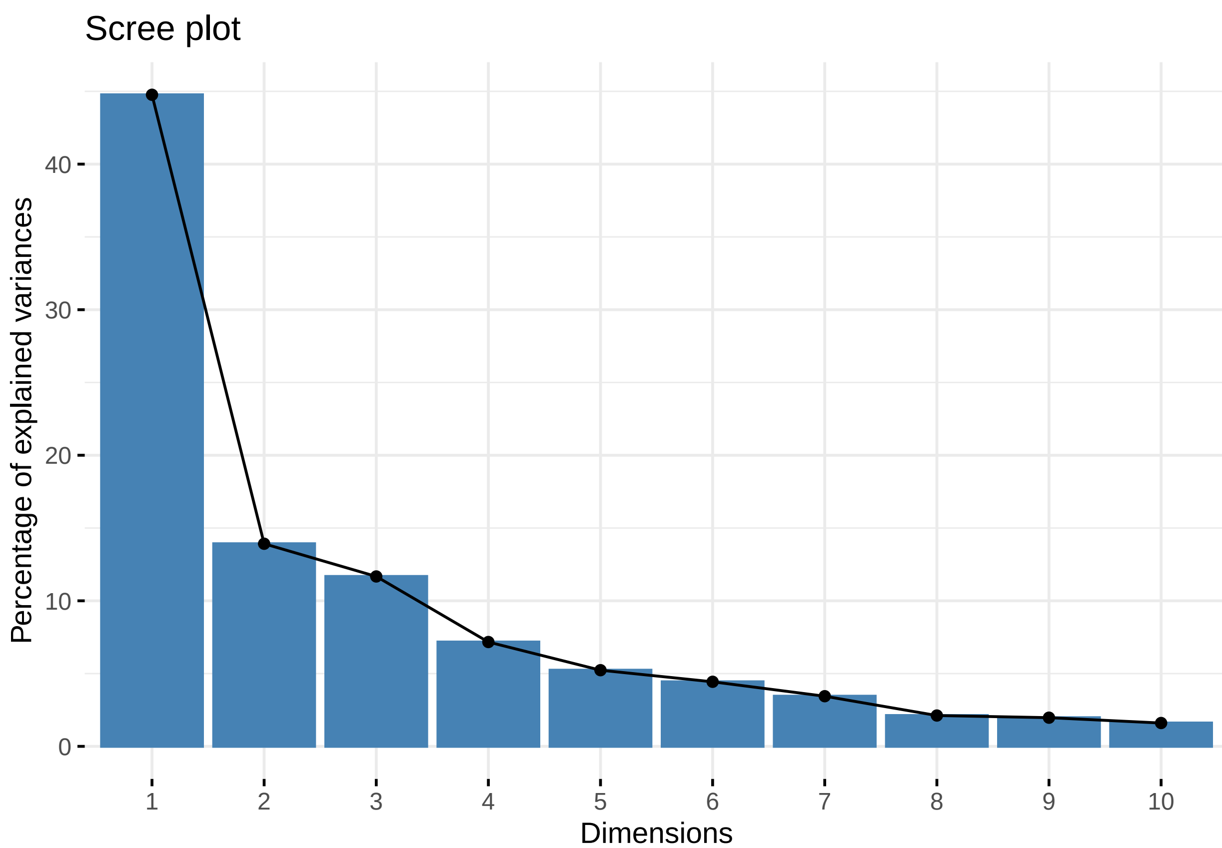
```r
# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- outfield_pca$eig[1, "percentage of variance"]
dim2_variance <- outfield_pca$eig[2, "percentage of variance"]

# Normalize so they sum to 1 (as relative weights)
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

```
## [1] "Dim.1 weight: 0.763"
```

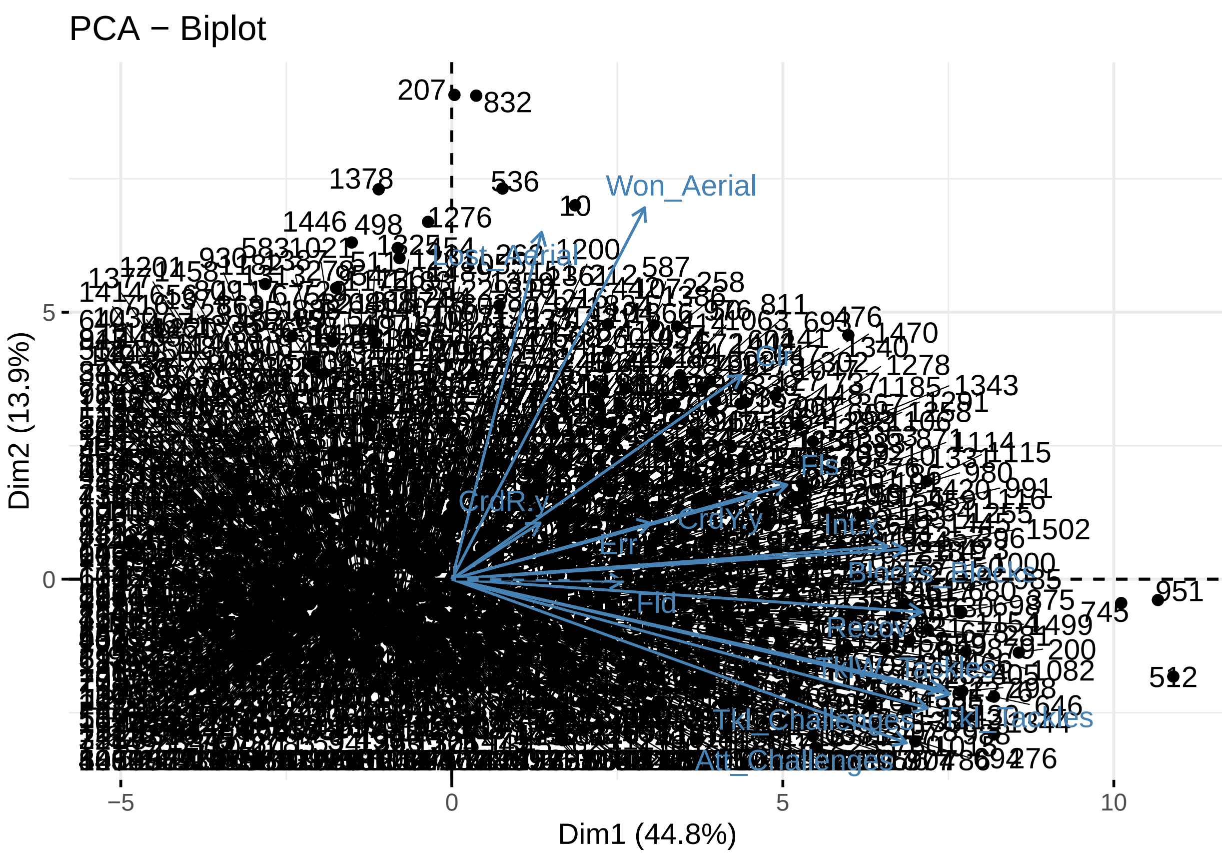
```r
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

```
## [1] "Dim.2 weight: 0.237"
```

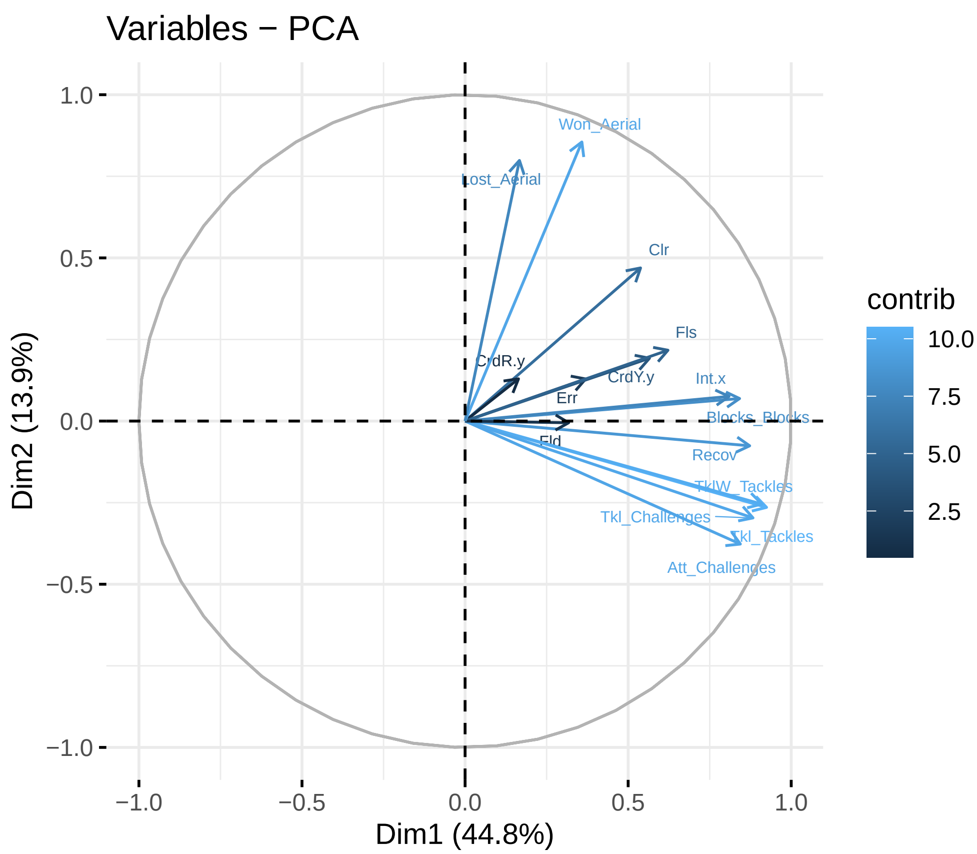
```r
# Plot the variance explained
fviz_eig(outfield_pca)
```




```r
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(outfield_pca, repel = TRUE)
```



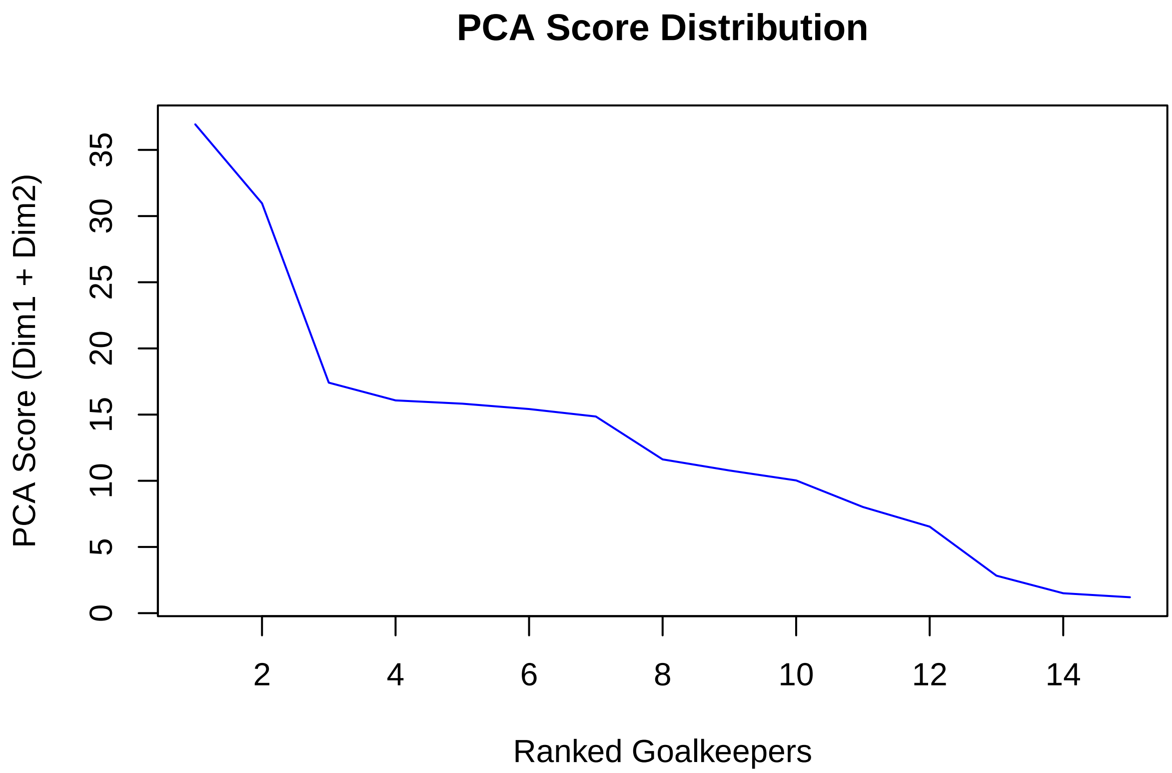

```r
# Contributions of variables to PC1 and PC2
fviz_pca_var(outfield_pca,
             col.var = "contrib",
             labelsize = 2, repel = TRUE)
```




```r
variable_contrib <- data.frame(outfield_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 15.94949
```

```r
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
     ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```



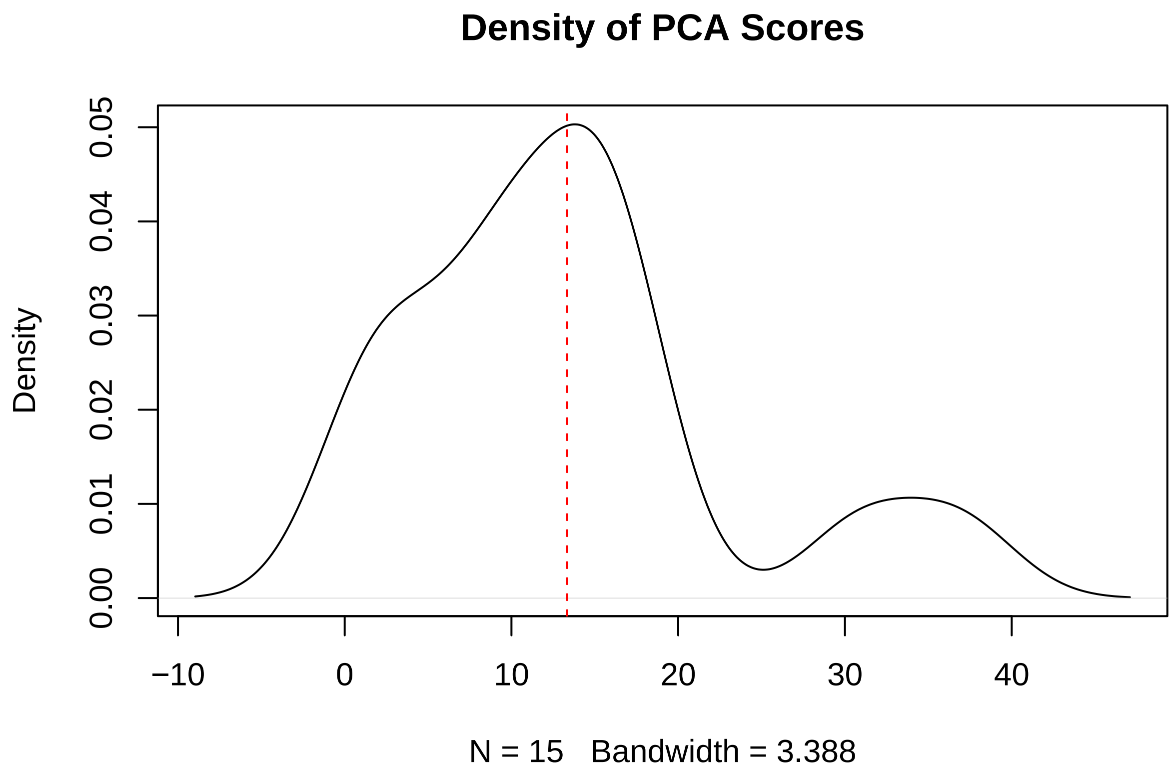

```r
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##     j1 j2 chi
## ESE  3 14 8.5
## EDE  3 15 9.0
```

```r
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 23.32218
```

```r
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```




```
#########
```

```
outfield_dpr <- outfield_data %>% mutate(Defensive_Player_Rtg = ((Tkl_Tackles* variable_contrib["Tkl_Ta


outfield_dpr_filtered <- outfield_dpr %>%
  filter(is.finite(Defensive_Player_Rtg), !is.na(Defensive_Player_Rtg))


#> Get the Average possession for every team in the top 5 leagues (will be useful for other metrics too
#> For defenders, divide DPR by (100 - Avg. Poss.)
#> First things first, let's extract the possession data at team level

big5_team_possession <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "possession", t
dplyr::glimpse(big5_team_possession)
```

```
## Rows: 196
## Columns: 30
## $ Season_End_Year     <int> 2023, 2023, 2023, 2023, 2023, 2023, 2023, 2023, 20~
## $ Squad               <chr> "Ajaccio", "Ajaccio", "Almería", "Almería", "Anger~
## $ Comp                <chr> "Ligue 1", "Ligue 1", "La Liga", "La Liga", "Ligue~
## $ Team_or_Opponent    <chr> "team", "opponent", "team", "opponent", "team", "o~
## $ Num_Players         <dbl> 36, 36, 29, 29, 33, 33, 26, 26, 26, 26, 27, 27, 26~
## $ Poss                <dbl> 43.2, 56.8, 45.1, 54.9, 46.9, 53.1, 59.3, 40.1, 49~
## $ Mins_Per_90         <dbl> 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38~
```

```
## $ Touches_Touches     <dbl> 19271, 24302, 20070, 23165, 21515, 23892, 25910, 1~
## $ `Def Pen_Touches`   <dbl> 1979, 1953, 2715, 1977, 2041, 1965, 1994, 2933, 27~
## $ `Def 3rd_Touches`   <dbl> 6134, 6843, 7746, 6271, 6588, 6828, 6632, 7798, 79~
## $ `Mid 3rd_Touches`   <dbl> 9380, 12450, 8468, 10606, 10633, 12114, 11453, 768~
## $ `Att 3rd_Touches`   <dbl> 3966, 5271, 4071, 6490, 4609, 5218, 8015, 3935, 47~
## $ `Att Pen_Touches`   <dbl> 486, 764, 631, 1018, 567, 824, 1296, 673, 746, 870~
## $ Live_Touches        <dbl> 19262, 24292, 20067, 23159, 21511, 23881, 25906, 1~
## $ Att_Take            <dbl> 632, 775, 666, 711, 928, 708, 758, 588, 682, 732, ~
## $ Succ_Take           <dbl> 285, 381, 296, 343, 441, 300, 358, 261, 293, 336, ~
## $ Succ_percent_Take   <dbl> 45.1, 49.2, 44.4, 48.2, 47.5, 42.4, 47.2, 44.4, 43~
## $ Tkld_Take           <dbl> 277, 313, 310, 280, 392, 310, 290, 249, 279, 312, ~
## $ Tkld_percent_Take   <dbl> 43.8, 40.4, 46.5, 39.4, 42.2, 43.8, 38.3, 42.3, 40~
## $ Carries_Carries     <dbl> 12563, 17674, 10792, 13816, 15521, 17872, 15923, 1~
## $ TotDist_Carries     <dbl> 73530, 100146, 56990, 75540, 89958, 102417, 83916,~
## $ PrgDist_Carries     <dbl> 32337, 48472, 29135, 41267, 41416, 48645, 44184, 2~
## $ PrgC_Carries        <dbl> 487, 636, 569, 860, 632, 725, 824, 510, 637, 608, ~
## $ Final_Third_Carries <dbl> 406, 558, 433, 626, 596, 589, 584, 354, 434, 425, ~
## $ CPA_Carries         <dbl> 92, 135, 131, 236, 149, 160, 281, 147, 174, 185, 2~
## $ Mis_Carries         <dbl> 625, 539, 580, 483, 593, 564, 526, 476, 565, 486, ~
## $ Dis_Carries         <dbl> 324, 324, 281, 277, 349, 342, 378, 319, 371, 320, ~
## $ Rec_Receiving       <dbl> 11114, 16224, 12047, 15280, 13974, 16318, 18101, 1~
## $ PrgR_Receiving      <dbl> 1150, 1685, 1080, 1692, 1374, 1589, 2024, 901, 122~
## $ Url                 <chr> "https://fbref.com/en/squads/7a54bb4f/2022-2023/Aj~
```

```
big5_team_possession_opponent <- big5_team_possession %>% filter(Team_or_Opponent == "opponent") %>% sel
big5_team_possession_team <- big5_team_possession %>% filter(Team_or_Opponent == "team") %>% select(Squa

outfield_dpr_filtered_possopp <- outfield_dpr_filtered %>% left_join(big5_team_possession_opponent, by =
outfield_dpr_filtered_possopp_possteam <- outfield_dpr_filtered_possopp %>% left_join(big5_team_possessi

outfield_DPR <- outfield_dpr_filtered_possopp_possteam %>% mutate(Poss.Adj.DPR = Defensive_Player_Rtg*10
avg_dpr <- mean(outfield_DPR$Poss.Adj.DPR)
outfield_DPR <- outfield_DPR %>% mutate(Poss.Adj.DPR = (Poss.Adj.DPR/avg_dpr)) %>% arrange(desc(Poss.Ad


pl_dpr <- outfield_DPR %>% filter(Comp == "Premier League")
ligue1_dpr <- outfield_DPR %>% filter(Comp == "Ligue 1")
seriea_dpr <- outfield_DPR %>% filter(Comp == "Serie A")
bundesliga_dpr <- outfield_DPR %>% filter(Comp == "Bundesliga")
laliga_dpr <- outfield_DPR %>% filter(Comp == "La Liga")

#> Outfield_DPR is what you have to carry forward because you cannot have players ineligible for any ca
```

Offensive Creation Rating

```
#>OFFENSIVE CREATION RATING

outfield_ocr_data <- outfield_data %>% select(Season_End_Year, Squad, Comp, Player, Nation, Pos, Age, Mi

colnames(outfield_ocr_data)[21] <- "Att_3rd_Touches"
colnames(outfield_ocr_data)[22] <- "Att_Pen_Touches"
```

```r
outfield_above_900 <- outfield_ocr_data%>% filter(Min_Playing.Time >= 900)
outfield_above_900_complete <- na.omit(outfield_above_900)

outfield_numeric <-outfield_above_900 %>% select(where(is.numeric))

# Optional: remove NAs
outfield_numeric <- outfield_numeric[, -1]
outfield_numeric <- na.omit(outfield_numeric)

# Scale the data (important for PCA since units differ)
outfield_scaled <- scale(outfield_numeric)
outfield_scaled <- outfield_scaled[ ,-1]
# Run PCA
outfield_pca <- PCA(outfield_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
outfield_pca$eig
```

```
##          eigenvalue percentage of variance cumulative percentage of variance
## comp 1  12.953863185           46.263797091                          46.26380
## comp 2   7.786126520           27.807594713                          74.07139
## comp 3   2.070498477            7.394637417                          81.46603
## comp 4   1.166181661            4.164934502                          85.63096
## comp 5   0.820809610            2.931462893                          88.56243
## comp 6   0.638103021            2.278939360                          90.84137
## comp 7   0.575255760            2.054484857                          92.89585
## comp 8   0.367293702            1.311763220                          94.20761
## comp 9   0.287714620            1.027552216                          95.23517
## comp 10  0.231268649            0.825959459                          96.06113
## comp 11  0.182263868            0.650942387                          96.71207
## comp 12  0.126831776            0.452970627                          97.16504
## comp 13  0.123160373            0.439858475                          97.60490
## comp 14  0.118549126            0.423389735                          98.02829
## comp 15  0.108470592            0.387394971                          98.41568
## comp 16  0.089530298            0.319751064                          98.73543
## comp 17  0.074983890            0.267799606                          99.00323
## comp 18  0.068047672            0.243027399                          99.24626
## comp 19  0.050995136            0.182125487                          99.42839
## comp 20  0.040342526            0.144080448                          99.57247
## comp 21  0.027465164            0.098089872                          99.67056
## comp 22  0.022338625            0.079780805                          99.75034
## comp 23  0.020162657            0.072009490                          99.82235
## comp 24  0.019845387            0.070876383                          99.89322
## comp 25  0.012474199            0.044550712                          99.93777
## comp 26  0.011043687            0.039441738                          99.97721
## comp 27  0.005029051            0.017960897                          99.99518
## comp 28  0.001350769            0.004824176                         100.00000
```

```r
# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- outfield_pca$eig[1, "percentage of variance"]
dim2_variance <- outfield_pca$eig[2, "percentage of variance"]

# Normalize so they sum to 1 (as relative weights)
```

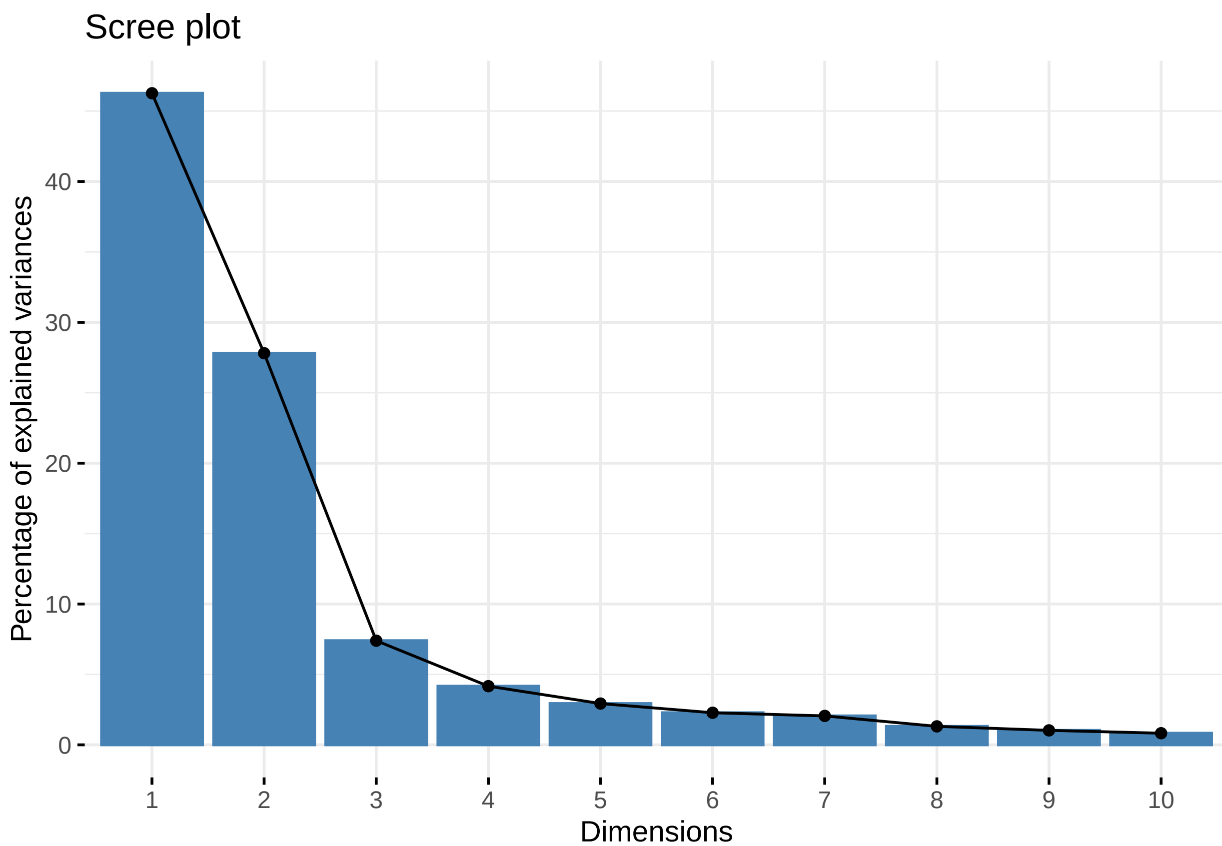
```r
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

```
## [1] "Dim.1 weight: 0.625"
```

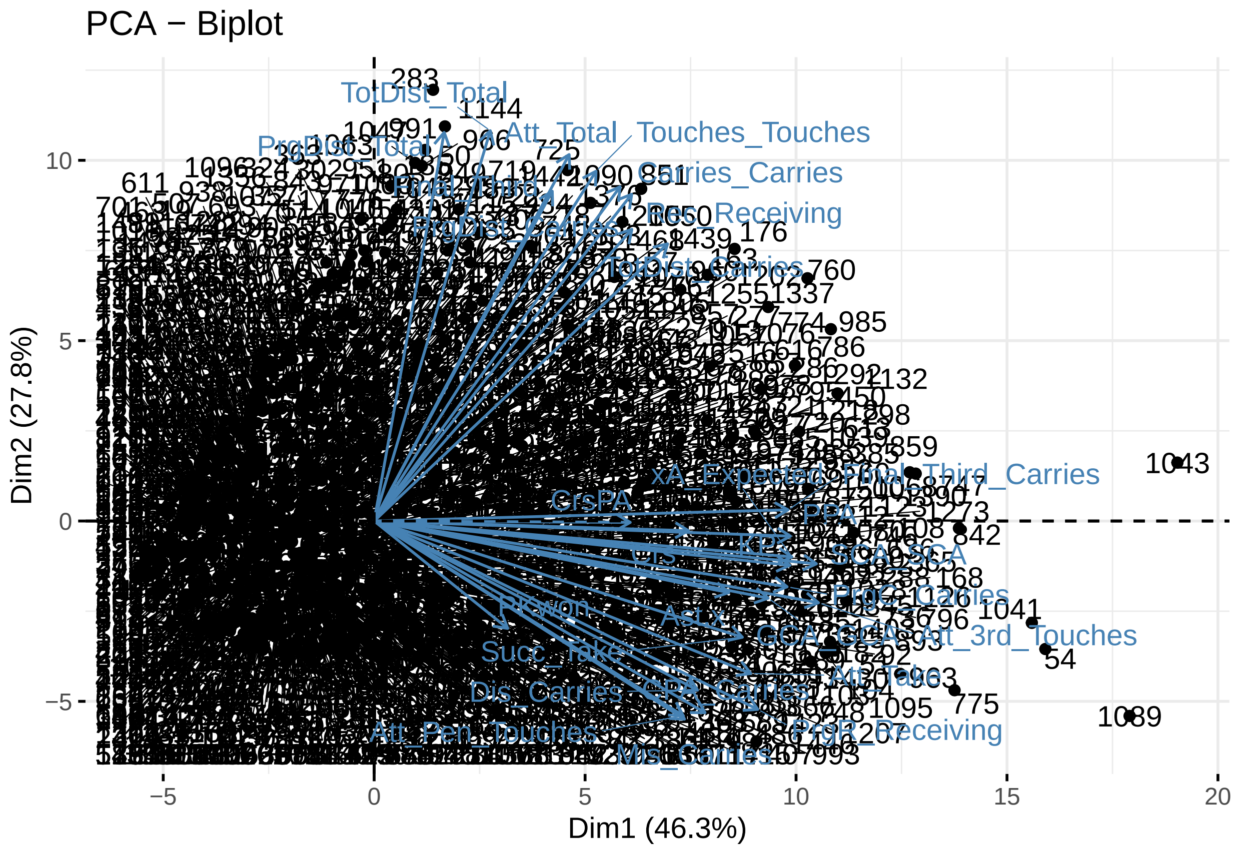
```r
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

```
## [1] "Dim.2 weight: 0.375"
```

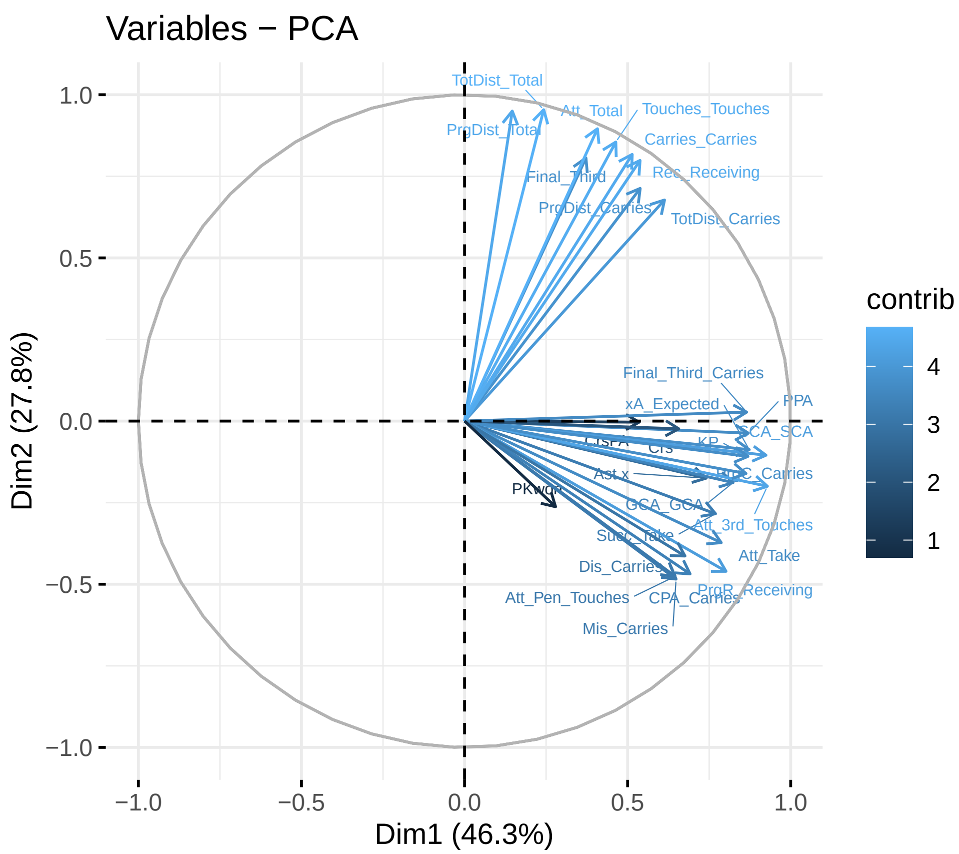
```r
# Plot the variance explained
fviz_eig(outfield_pca)
```




```r
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(outfield_pca, repel = TRUE)
```



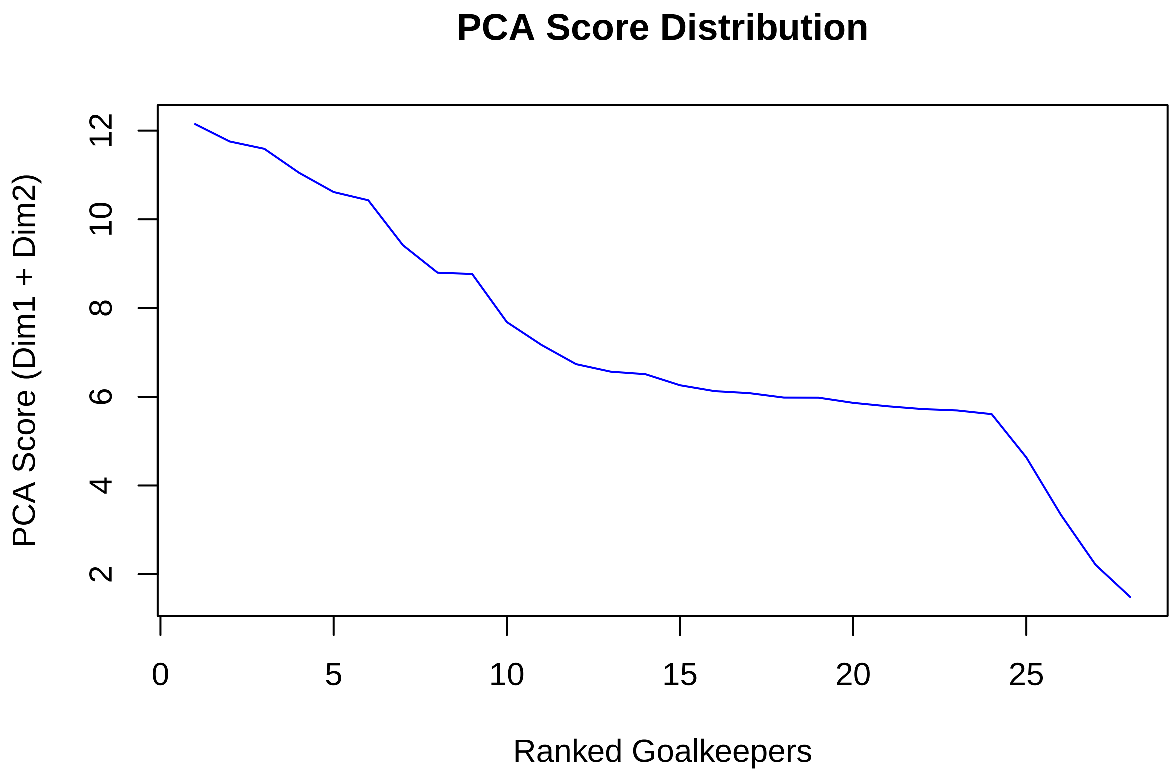

```r
# Contributions of variables to PC1 and PC2
fviz_pca_var(outfield_pca,
             col.var = "contrib",
             labelsize = 2, repel = TRUE)
```




```r
variable_contrib <- data.frame(outfield_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 8.952647
```

```r
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
     ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```



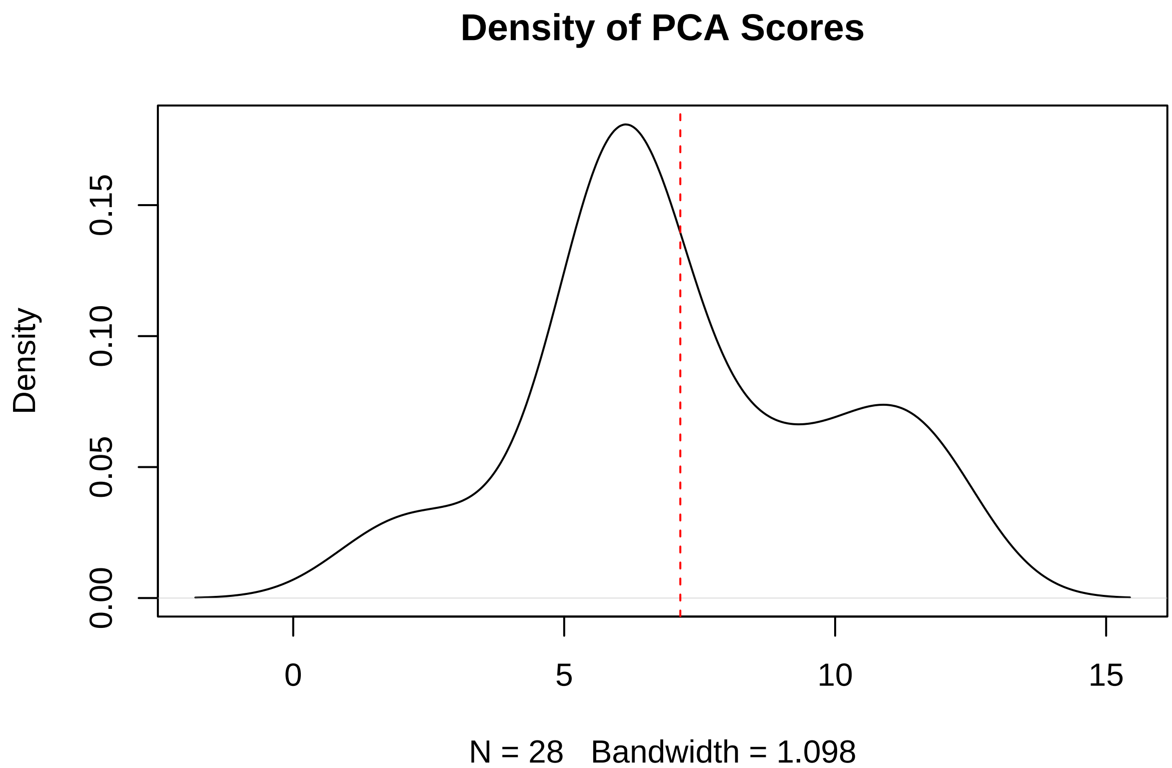

```
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##     j1 j2 chi
## ESE 12 24  18
## EDE 12 24  18
```

```
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 9.922581
```

```
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```




```
###########
```

```
#> OFFENSIVE CREATION RATING

outfield_DPR_ocr <- outfield_above_900 %>% mutate(Off_Creation_Rtg = (((Ast.x *variable_contrib["Ast.x"

setdiff(c("Ast.x", "xA_Expected", "SCA_SCA", "GCA_GCA", "KP", "Final_Third", "PPA", "Att_Total", "CrsPA
```

```
## character(0)
```

```
big5_team_possession <- fb_big5_advanced_season_stats(season_end_year= 2023, stat_type= "possession", t
dplyr::glimpse(big5_team_possession)
```

```
## Rows: 196
## Columns: 30
## $ Season_End_Year     <int> 2023, 2023, 2023, 2023, 2023, 2023, 2023, 2023, 20~
## $ Squad               <chr> "Ajaccio", "Ajaccio", "Almería", "Almería", "Anger~
## $ Comp                <chr> "Ligue 1", "Ligue 1", "La Liga", "La Liga", "Ligue~
## $ Team_or_Opponent    <chr> "team", "opponent", "team", "opponent", "team", "o~
## $ Num_Players         <dbl> 36, 36, 29, 29, 33, 33, 26, 26, 26, 26, 27, 27, 26~
## $ Poss                <dbl> 43.2, 56.8, 45.1, 54.9, 46.9, 53.1, 59.3, 40.1, 49~
## $ Mins_Per_90         <dbl> 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38~
## $ Touches_Touches     <dbl> 19271, 24302, 20070, 23165, 21515, 23892, 25910, 1~
## $ `Def Pen_Touches`   <dbl> 1979, 1953, 2715, 1977, 2041, 1965, 1994, 2933, 27~
```

```
## $ `Def 3rd_Touches`   <dbl> 6134, 6843, 7746, 6271, 6588, 6828, 6632, 7798, 79~
## $ `Mid 3rd_Touches`   <dbl> 9380, 12450, 8468, 10606, 10633, 12114, 11453, 768~
## $ `Att 3rd_Touches`   <dbl> 3966, 5271, 4071, 6490, 4609, 5218, 8015, 3935, 47~
## $ `Att Pen_Touches`   <dbl> 486, 764, 631, 1018, 567, 824, 1296, 673, 746, 870~
## $ Live_Touches        <dbl> 19262, 24292, 20067, 23159, 21511, 23881, 25906, 1~
## $ Att_Take            <dbl> 632, 775, 666, 711, 928, 708, 758, 588, 682, 732, ~
## $ Succ_Take           <dbl> 285, 381, 296, 343, 441, 300, 358, 261, 293, 336, ~
## $ Succ_percent_Take   <dbl> 45.1, 49.2, 44.4, 48.2, 47.5, 42.4, 47.2, 44.4, 43~
## $ Tkld_Take           <dbl> 277, 313, 310, 280, 392, 310, 290, 249, 279, 312, ~
## $ Tkld_percent_Take   <dbl> 43.8, 40.4, 46.5, 39.4, 42.2, 43.8, 38.3, 42.3, 40~
## $ Carries_Carries     <dbl> 12563, 17674, 10792, 13816, 15521, 17872, 15923, 1~
## $ TotDist_Carries     <dbl> 73530, 100146, 56990, 75540, 89958, 102417, 83916,~
## $ PrgDist_Carries     <dbl> 32337, 48472, 29135, 41267, 41416, 48645, 44184, 2~
## $ PrgC_Carries        <dbl> 487, 636, 569, 860, 632, 725, 824, 510, 637, 608, ~
## $ Final_Third_Carries <dbl> 406, 558, 433, 626, 596, 589, 584, 354, 434, 425, ~
## $ CPA_Carries         <dbl> 92, 135, 131, 236, 149, 160, 281, 147, 174, 185, 2~
## $ Mis_Carries         <dbl> 625, 539, 580, 483, 593, 564, 526, 476, 565, 486, ~
## $ Dis_Carries         <dbl> 324, 324, 281, 277, 349, 342, 378, 319, 371, 320, ~
## $ Rec_Receiving       <dbl> 11114, 16224, 12047, 15280, 13974, 16318, 18101, 1~
## $ PrgR_Receiving      <dbl> 1150, 1685, 1080, 1692, 1374, 1589, 2024, 901, 122~
## $ Url                 <chr> "https://fbref.com/en/squads/7a54bb4f/2022-2023/Aj~
```

```r
big5_team_possession_opponent <- big5_team_possession %>% filter(Team_or_Opponent == "opponent") %>% sel
big5_team_possession_team <- big5_team_possession %>% filter(Team_or_Opponent == "team") %>% select(Squa

outfield_DPR_ocr <- outfield_DPR_ocr %>% left_join(big5_team_possession_opponent, by = "Squad", relatio
outfield_DPR_ocr <- outfield_DPR_ocr %>% left_join(big5_team_possession_team, by = "Squad", relationshi

outfield_DPR_ocr_possadj <- outfield_DPR_ocr %>% mutate(Poss.adj.OCR = (Off_Creation_Rtg/Poss_Team)*100
outfield_DPR_ocr_possadj <- outfield_DPR_ocr_possadj %>% filter(Poss.adj.OCR != "NaN")
avg_ocr <- mean(outfield_DPR_ocr_possadj$Poss.adj.OCR)
outfield_DPR_OCR <- outfield_DPR_ocr_possadj %>% mutate(Poss.adj.OCR = Poss.adj.OCR/avg_ocr) %>% arrang

outfield_DPR_OCR_5ast <- outfield_DPR_OCR %>% filter(Ast.x >= 5)
```

Goalscorer Rating

```r
#>GOALSCORER RATING

outfield_gsr_data <- outfield_data %>%
  mutate(
    Goals_per_xG = ifelse(xG_Expected.x > 0, Gls_Standard / xG_Expected.x, 0),
    G_per_Shot = ifelse(Sh_Standard > 0, Gls_Standard / Sh_Standard, 0),
    xG_per_Shot = ifelse(Sh_Standard > 0, xG_Expected.x / Sh_Standard, 0))


outfield_gsr_data <- outfield_gsr_data %>% mutate(Gls_per_xG = Gls/xG_Expected.x) %>% select(Season_End

colnames(outfield_gsr_data)[10] <- "xG_Expected"

outfield_above_900 <- outfield_gsr_data%>% filter(Min_Playing.Time >= 900)
outfield_above_900_complete <- na.omit(outfield_above_900)
```

```r
outfield_numeric <-outfield_above_900 %>% select(where(is.numeric))

# Optional: remove NAs
outfield_numeric <- outfield_numeric[, -1]
outfield_numeric <- na.omit(outfield_numeric)

# Scale the data (important for PCA since units differ)
outfield_scaled <- scale(outfield_numeric)
outfield_scaled <- outfield_scaled[ ,-1]
# Run PCA
outfield_pca <- PCA(outfield_scaled, graph = FALSE)

# View eigenvalues (variance explained by each component)
outfield_pca$eig
```

```
##           eigenvalue percentage of variance cumulative percentage of variance
## comp 1  5.943710e+00           5.403372e+01                          54.03372
## comp 2  2.467567e+00           2.243243e+01                          76.46615
## comp 3  1.038757e+00           9.443242e+00                          85.90939
## comp 4  6.647987e-01           6.043625e+00                          91.95302
## comp 5  4.315585e-01           3.923259e+00                          95.87628
## comp 6  1.881929e-01           1.710844e+00                          97.58712
## comp 7  1.558292e-01           1.416629e+00                          99.00375
## comp 8  6.715207e-02           6.104733e-01                          99.61422
## comp 9  3.695575e-02           3.359614e-01                          99.95018
## comp 10 5.479679e-03           4.981526e-02                         100.00000
## comp 11 1.333712e-33           1.212465e-32                         100.00000
```

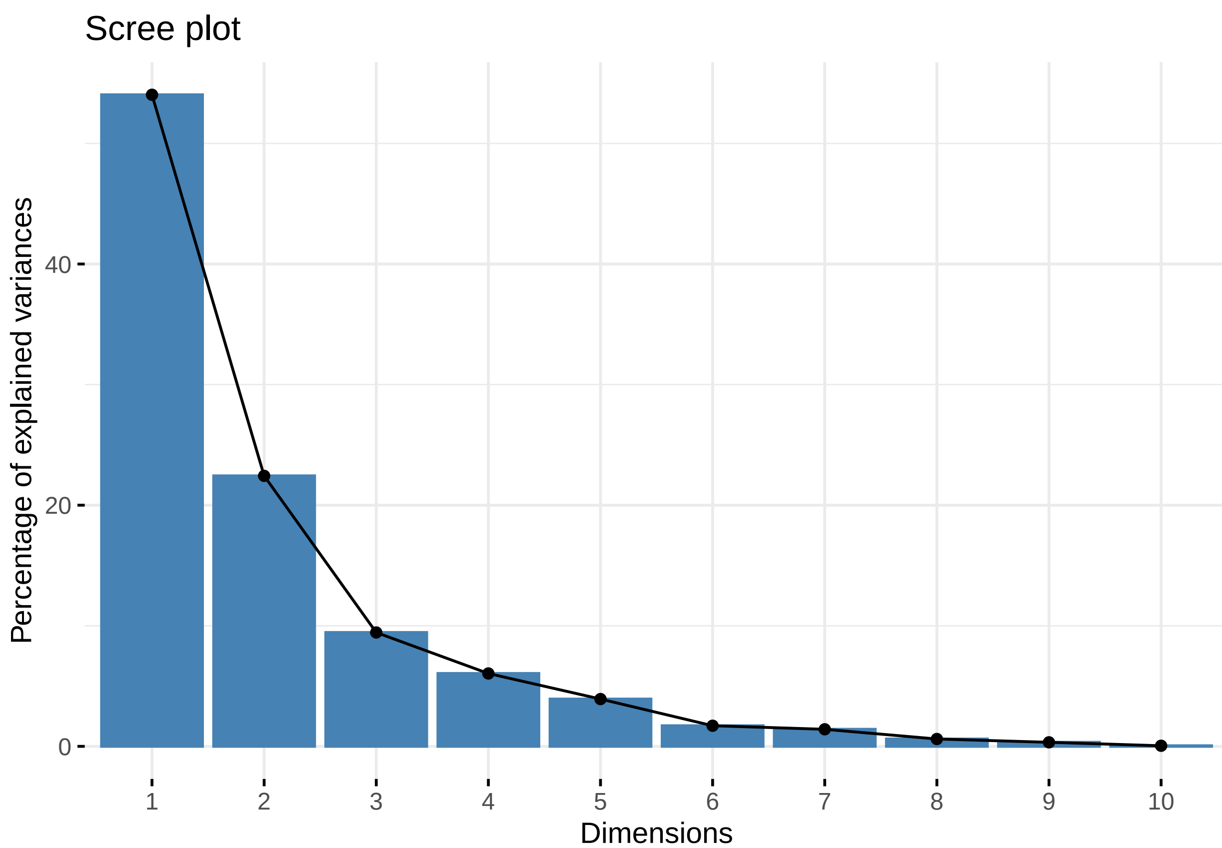
```r
# Extract variance explained by Dim.1 and Dim.2
dim1_variance <- outfield_pca$eig[1, "percentage of variance"]
dim2_variance <- outfield_pca$eig[2, "percentage of variance"]

# Normalize so they sum to 1 (as relative weights)
total_variance <- dim1_variance + dim2_variance
dim1_weight <- dim1_variance / total_variance
dim2_weight <- dim2_variance / total_variance

print(paste("Dim.1 weight:", round(dim1_weight, 3)))
```

```
## [1] "Dim.1 weight: 0.707"
```

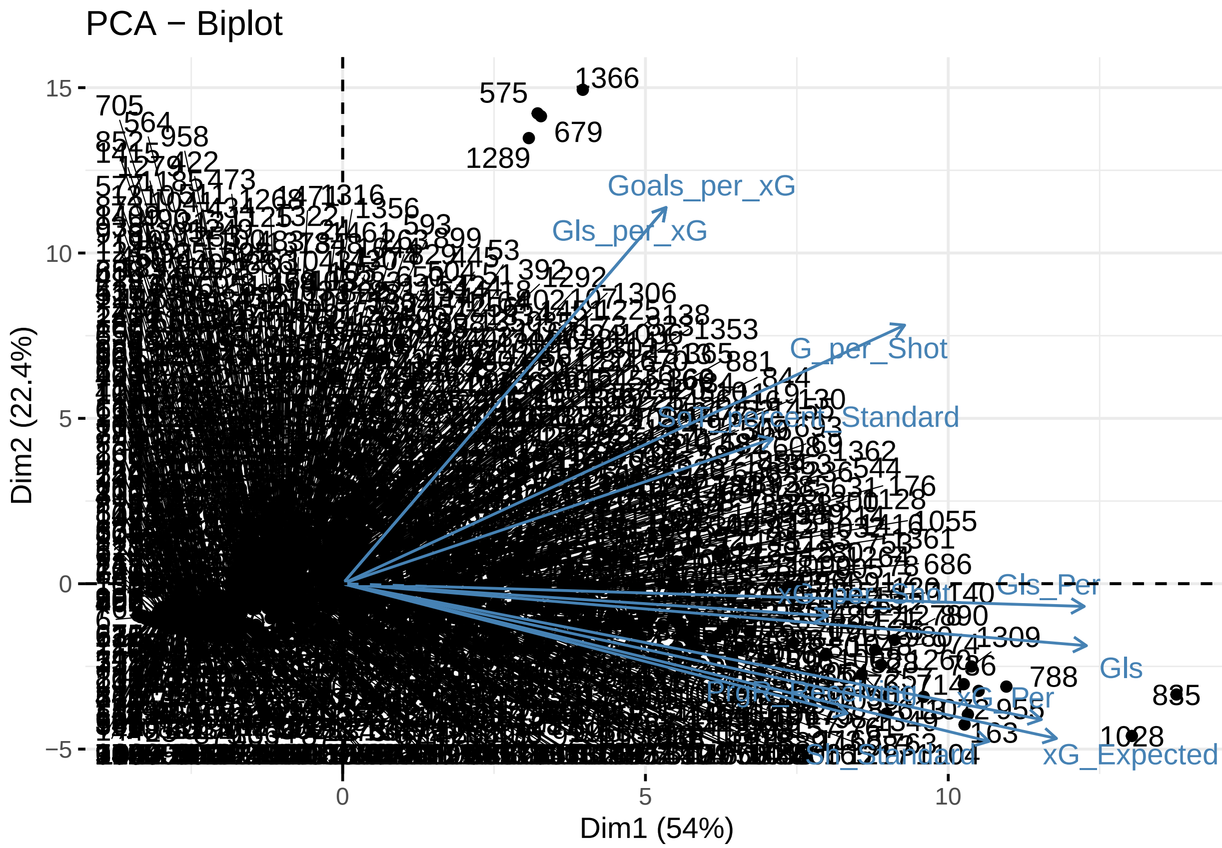
```r
print(paste("Dim.2 weight:", round(dim2_weight, 3)))
```

```
## [1] "Dim.2 weight: 0.293"
```

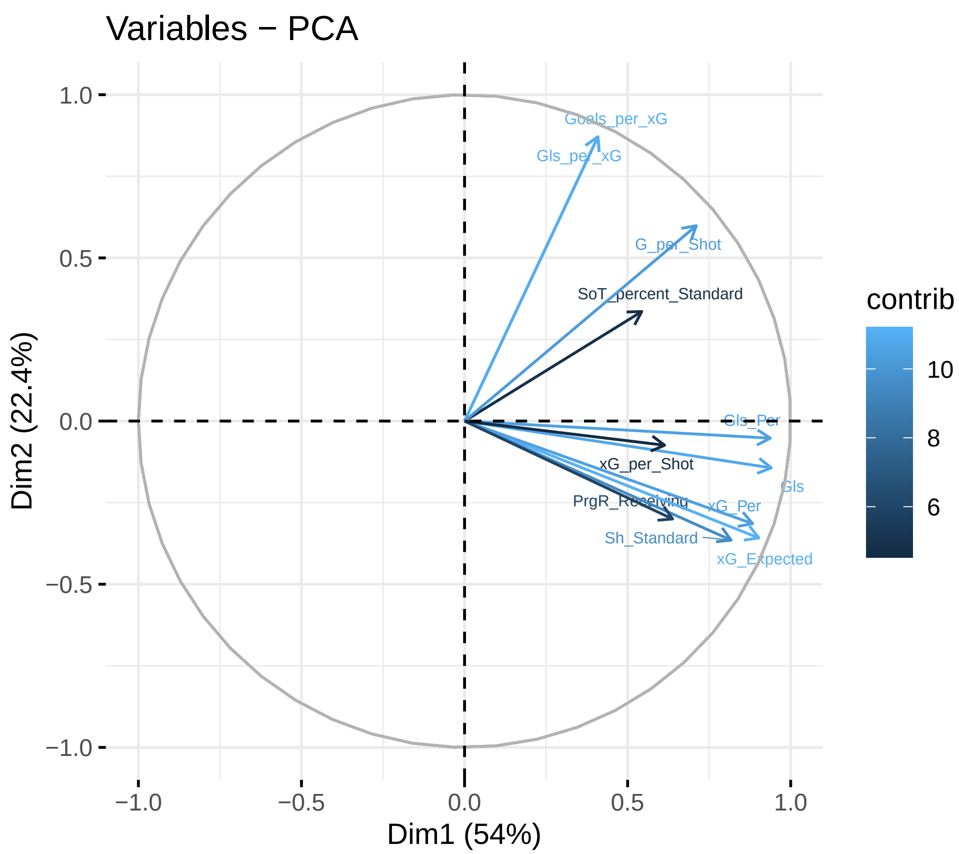
```r
# Plot the variance explained
fviz_eig(outfield_pca)
```




```r
# Biplot (players and variables on first two PCs)
fviz_pca_biplot(outfield_pca, repel = TRUE)
```



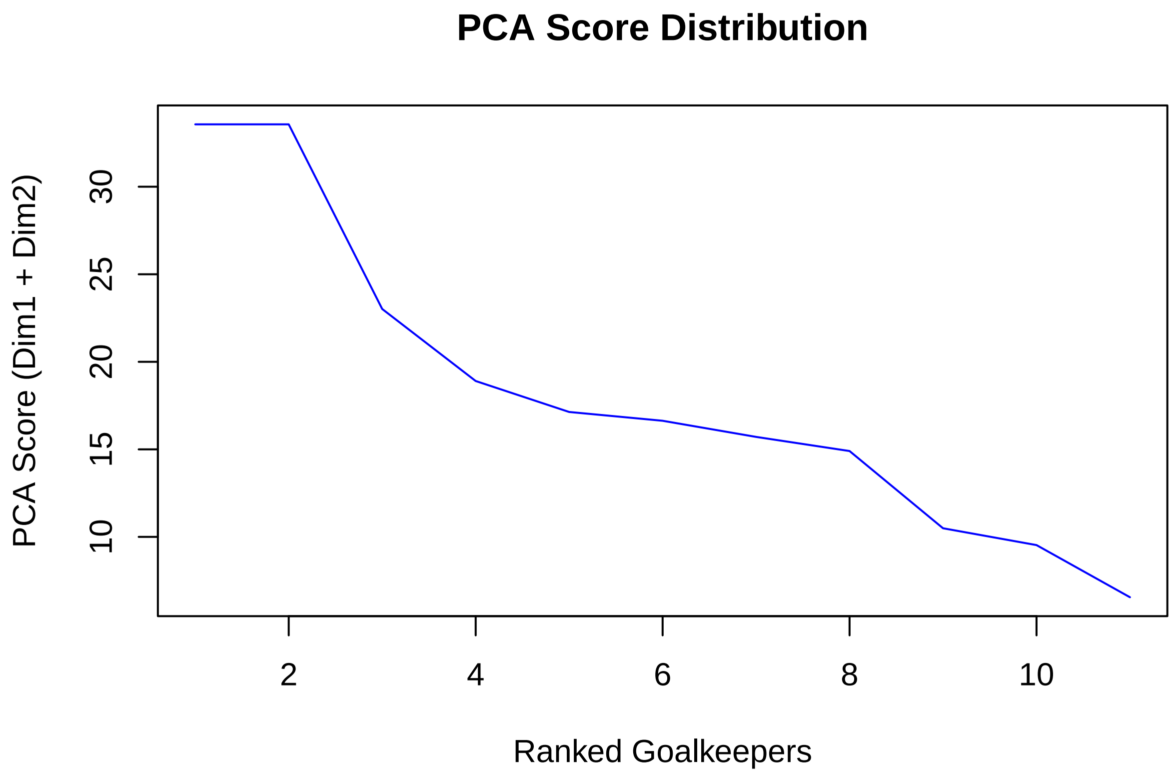

```r
# Contributions of variables to PC1 and PC2
fviz_pca_var(outfield_pca,
             col.var = "contrib",
             labelsize = 2, repel = TRUE)
```




```
variable_contrib <- data.frame(outfield_pca$var$contrib)
variable_contrib <- variable_contrib %>%
  mutate(Total_Contrib = Dim.1 + Dim.2)

# Scale them to sum to 1 (to act as relative weights)
variable_contrib <- variable_contrib %>%
  mutate(Weight = Total_Contrib / sum(Total_Contrib))

# 75th percentile threshold
threshold_75 <- quantile(variable_contrib$Total_Contrib, 0.75)
print(threshold_75)
```

```
##      75%
## 20.95959
```

```
# Sorted plot
sorted_scores <- sort(variable_contrib$Total_Contrib, decreasing = TRUE)
plot(sorted_scores, type = "l", col = "blue", main = "PCA Score Distribution",
     ylab = "PCA Score (Dim1 + Dim2)", xlab = "Ranked Goalkeepers")
```



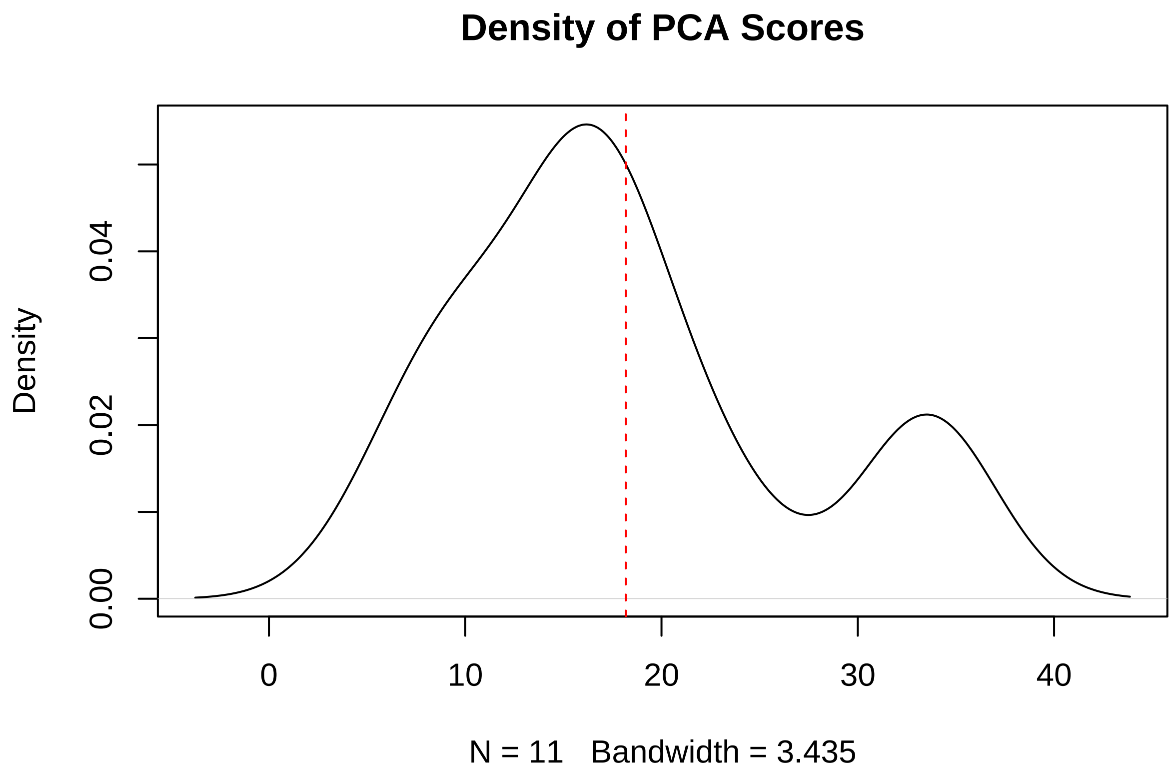

```r
# Find the elbow point
elbow <- inflection::findiplist(1:length(sorted_scores), sorted_scores, 0)
print(elbow)
```

```
##     j1 j2 chi
## ESE  5 10 7.5
## EDE  4  2 NaN
```

```r
mean_score <- mean(variable_contrib$Total_Contrib)
sd_score <- sd(variable_contrib$Total_Contrib)

threshold_sd <- mean_score + sd_score
print(threshold_sd)
```

```
## [1] 27.05011
```

```r
plot(density(variable_contrib$Total_Contrib), main = "Density of PCA Scores")
abline(v = mean_score, col = "red", lty = 2)
```




```
###########
```

```
#> GOALSCORER RATING
#Verify if the goals are per 90 or per shot - think they might be per 90
outfield_DPR_OCR_gsr <- outfield_gsr_data %>% filter(Min_Playing.Time > 900) %>% mutate(GoalScorer_Rtg =
    (ifelse(xG_Expected > 0, Gls / xG_Expected, 0) * variable_contrib["Gls_per_xG", "Weight"]) +
    (SoT_percent_Standard * variable_contrib["SoT_percent_Standard", "Weight"]) +
    (ifelse(Sh_Standard > 0, Gls / Sh_Standard, 0) * variable_contrib["G_per_Shot", "Weight"]) +
    (xG_per_Shot * variable_contrib["xG_per_Shot", "Weight"]) +
    (PrgR_Receiving * variable_contrib["PrgR_Receiving", "Weight"])
) / Min_Playing.Time) %>% arrange(desc(GoalScorer_Rtg))


outfield_gsr_awards <- outfield_DPR_OCR_gsr %>% filter(Min_Playing.Time >= 2160)
outfield_GSR <- outfield_DPR_OCR_gsr
```

```
#Correlation work between GA and GKR

cor(outfield_GSR$GoalScorer_Rtg, outfield_GSR$Gls_Per, use = "complete.obs", method = "spearman")
```

Sanity check GSR with xG

```
## [1] 0.6591302
```

```r
cor(outfield_GSR$GoalScorer_Rtg, outfield_GSR$xG_Per, use = "complete.obs", method = "spearman")
```

```
## [1] 0.673855
```

```r
summary_model_GSR <- lm(Gls_Per ~ GoalScorer_Rtg, data = outfield_GSR)
summary(summary_model_GSR)$r.squared
```

```
## [1] 0.3782466
```

Reassigning a few table names

```r
outfield_OCR <- outfield_DPR_OCR
```

Overall Player Ratings

```r


outfield_DPR_filtered <- outfield_DPR %>% select(Squad, Comp, Player, Pos, Poss.Adj.DPR, Min_Playing.Ti
outfield_OCR_filtered <- outfield_OCR %>% select(Squad, Comp, Player, Pos, Poss.adj.OCR, Min_Playing.Ti
outfield_GSR_filtered <- outfield_GSR %>% select(Squad, Comp, Player, Pos, GoalScorer_Rtg, Min_Playing.

player_master <- full_join(outfield_DPR_filtered, outfield_OCR_filtered, by = "Player") %>% full_join(o
```

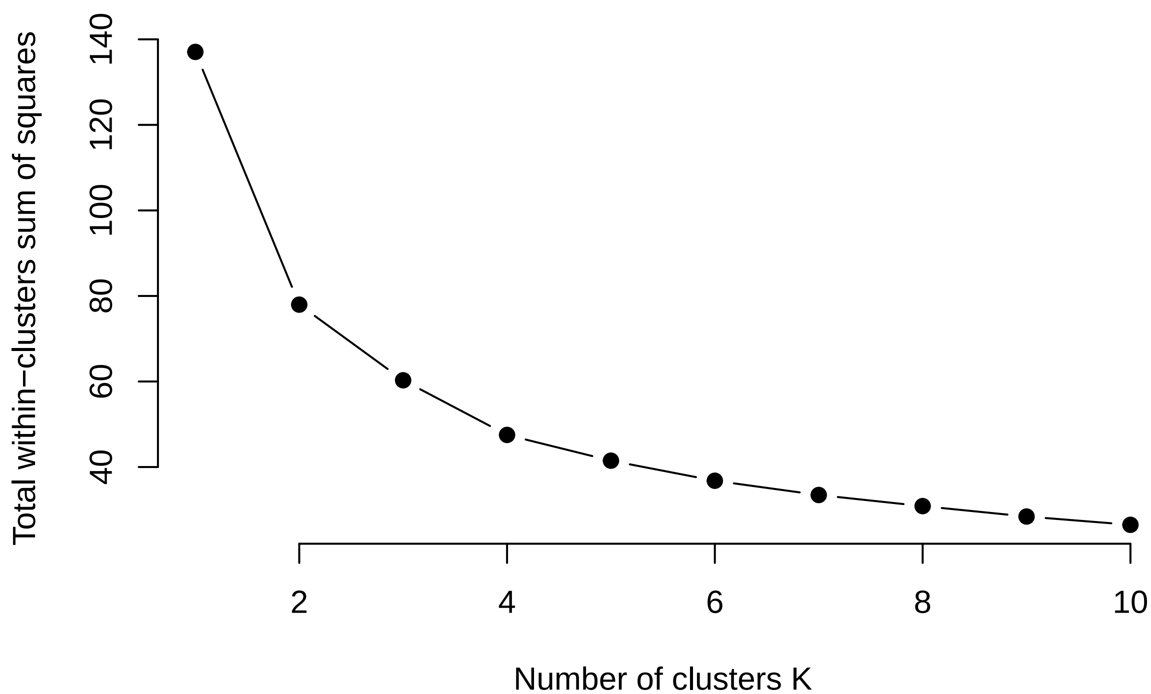
Take the four ratings dataframes and join them before doing clustering on them

```
## Warning in full_join(outfield_DPR_filtered, outfield_OCR_filtered, by = "Player"): Detected an unexpe
## i Row 1 of `x` matches multiple rows in `y`.
## i Row 115 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship =
##   "many-to-many"` to silence this warning.

## Warning in full_join(., outfield_GSR_filtered, by = "Player"): Detected an unexpected many-to-many r
## i Row 1 of `x` matches multiple rows in `y`.
## i Row 371 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship =
##   "many-to-many"` to silence this warning.
```

```r
player_master <- player_master %>%
  mutate(
    DPR_std = (Poss.Adj.DPR - min(Poss.Adj.DPR, na.rm=TRUE)) / (max(Poss.Adj.DPR, na.rm=TRUE) - min(Poss
    OCR_std = (Poss.adj.OCR - min(Poss.adj.OCR, na.rm=TRUE)) / (max(Poss.adj.OCR, na.rm=TRUE) - min(Poss
    GSR_std = (GoalScorer_Rtg - min(GoalScorer_Rtg, na.rm=TRUE)) / (max(GoalScorer_Rtg, na.rm=TRUE) - mi
  )

player_master <- player_master %>%  mutate(
    Squad= coalesce(Squad, Squad.x, Squad.y),
    Comp= coalesce(Comp, Comp.x, Comp.y),
    Pos = coalesce(Pos.x, Pos.y, Pos),
    Min_Playing_Time = coalesce(Min_Playing.Time.x, Min_Playing.Time.y, Min_Playing.Time))

player_master_final <- player_master %>% select(Player, Squad, Comp, Pos, Poss.Adj.DPR, Poss.adj.OCR, G
```

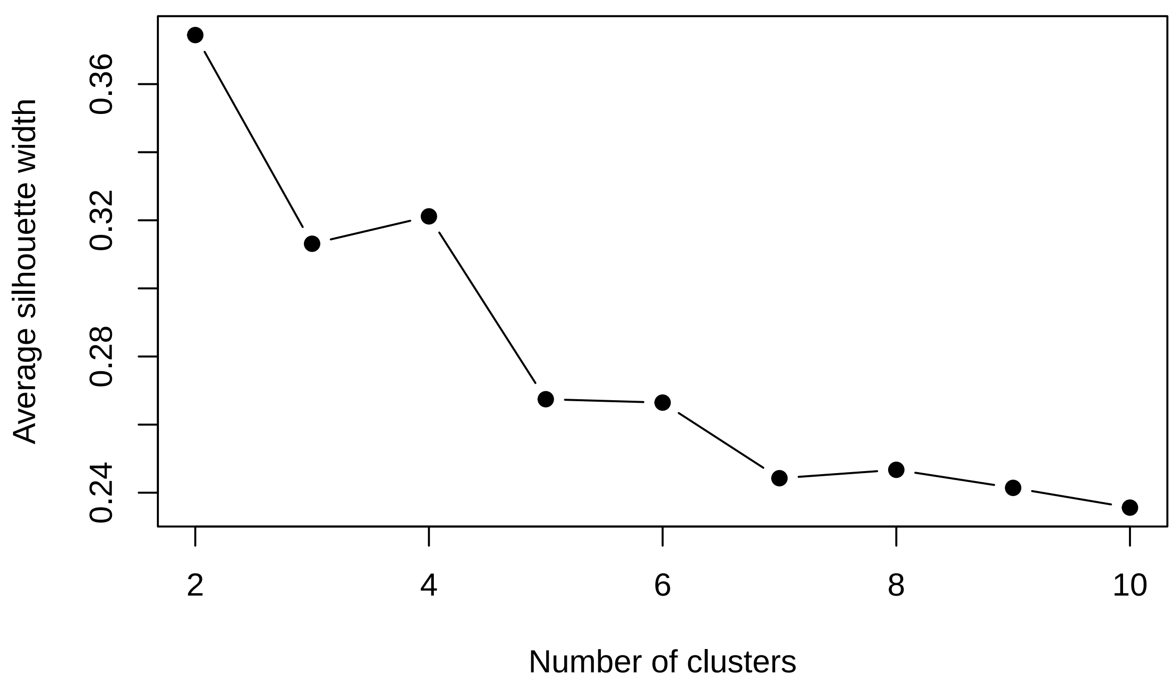
Cluster the players to look for any natural clusterings then run PCA on each cluster

```
player_clustering <- player_master_final %>%
  select(DPR_std, OCR_std, GSR_std) %>%
  mutate(across(everything(), ~ ifelse(is.na(.), 0, .)))

wss <- function(k) {
  kmeans(player_clustering, k, nstart = 25)$tot.withinss
}

k.values <- 1:10
wss_values <- map_dbl(k.values, wss)

plot(k.values, wss_values, type = "b",
     pch = 19, frame = FALSE,
     xlab = "Number of clusters K",
     ylab = "Total within-clusters sum of squares")
```




```
library(cluster)

sil_width <- function(k){
  pam(player_clustering, k)$silinfo$avg.width
}
```

```r
k.values <- 2:10
sil_values <- map_dbl(k.values, sil_width)

plot(k.values, sil_values, type = "b", pch = 19,
     xlab = "Number of clusters",
     ylab = "Average silhouette width")
```




```r
set.seed(212)
kmeans_result <- kmeans(player_clustering, centers = 2, nstart = 25)

player_master_final$Cluster <- kmeans_result$cluster
```

```r
run_pca_weights <- function(data_numeric){
  # Perform PCA
  pca <- PCA(data_numeric, graph = FALSE)

  # Extract eigenvalues (variance explained by each component)
  eig <- pca$eig

  # Extract contributions of variables to PC1 and PC2
  var_contrib <- data.frame(pca$var$contrib)

  # Sum contribution of Dim.1 and Dim.2 and scale them to sum to 1
  var_contrib <- var_contrib %>%
    mutate(Total_Contrib = Dim.1 + Dim.2,
```

```
        Weight = Total_Contrib / sum(Total_Contrib))

  # Return PCA object, weights, and eigenvalues
  return(list(pca = pca, weights = var_contrib, eig = eig))
}

player_clustering_final <- player_clustering
player_clustering_final$Player <- player_master_final$Player
player_clustering_final$Squad <- player_master_final$Squad
player_clustering_final$Comp <- player_master_final$Comp
player_clustering_final$Pos <- player_master_final$Pos
player_clustering_final$Cluster <- player_master_final$Cluster
player_clustering_final$Min_Playing_Time <- player_master_final$Min_Playing_Time


clustered_pca_results <- player_clustering_final %>%
  group_split(Cluster) %>%
  map(~run_pca_weights(.x %>% select(DPR_std, OCR_std, GSR_std)))
```

```
cluster_weights <- map(clustered_pca_results, ~.x$weights %>% select(Weight) %>% rownames_to_column("Me
names(cluster_weights) <- paste0("Cluster_", 1:length(cluster_weights))

player_clusters <- player_clustering_final %>% group_split(Cluster)

cluster_ratings <- map2(
  player_clusters,
  cluster_weights,
  ~ .x %>% mutate(
    Overall_Rating =
      DPR_std * .y$Weight[.y$Metric == "DPR_std"] +
      OCR_std * .y$Weight[.y$Metric == "OCR_std"] +
      GSR_std * .y$Weight[.y$Metric == "GSR_std"]
  )
)

player_master_final <- bind_rows(cluster_ratings)
player_master_final <- player_master_final %>% arrange(desc(Overall_Rating))

pl_ratings_final <- player_master_final %>% filter(Comp == "Premier League")
laliga_ratings_final <- player_master_final %>% filter(Comp == "La Liga")
ligue1_ratings_final <- player_master_final %>% filter(Comp == "Ligue 1")
bundesliga_ratings_final <- player_master_final %>% filter(Comp == "Bundesliga")
seriea_ratings_final <- player_master_final %>% filter(Comp == "Serie A")
```

Finally decide the awards and the team of the year based on minutes - 2260 minutes played (24 games)

```
player_master_awards <- player_master_final %>% filter(Min_Playing_Time >= 2260)
pl_ratings_awards <- pl_ratings_final %>% filter(Min_Playing_Time >= 2260)
laliga_ratings_awards <- laliga_ratings_final %>% filter(Min_Playing_Time >= 2260)
ligue1_ratings_awards <- ligue1_ratings_final %>% filter(Min_Playing_Time >= 2260)
```

```r
seriea_ratings_awards <- seriea_ratings_final %>% filter(Min_Playing_Time >= 2260)
bundesliga_ratings_awards <- bundesliga_ratings_final %>% filter(Min_Playing_Time >= 2260)
```

Overall Team Ratings

```r
team_ratings_weighted <- player_master_final %>%
  group_by(Squad, Comp) %>%
  summarise(
    Weighted_Team_Rating = sum(Overall_Rating * Min_Playing_Time, na.rm = TRUE) / sum(Min_Playing_Time,
    Total_Minutes = sum(Min_Playing_Time, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(Weighted_Team_Rating))


pl_team_ratings <- team_ratings_weighted %>% filter(Comp == "Premier League")
laliga_team_ratings <- team_ratings_weighted %>% filter(Comp == "La Liga")
ligue1_team_ratings <- team_ratings_weighted %>% filter(Comp == "Ligue 1")
bundesliga_team_ratings <- team_ratings_weighted %>% filter(Comp == "Bundesliga")
seriea_team_ratings <- team_ratings_weighted %>% filter(Comp == "Serie A")
```

No Keeper Ratings

```r
pl_ratings_table <- pl_team_ratings %>% select(Squad, Weighted_Team_Rating)
pl_table <- prem_league_table %>% select(Squad) %>% rownames_to_column()
colnames(pl_table)[1] <- "Position"
pl_xg_table <- prem_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

pl_combined_table <- pl_ratings_table %>%
  left_join(pl_table, by = c("Squad")) %>%
  left_join(pl_xg_table, by = c("Squad"))

pl_combined_table$Position <- as.numeric(pl_combined_table$Position)

cor(pl_combined_table$Weighted_Team_Rating, pl_combined_table$Position, method = "spearman")
```

Premier League

```
## [1] -0.8225564
```

```r
cor(pl_combined_table$Weighted_Team_Rating, pl_combined_table$Pts, method = "spearman")
```

```
## [1] 0.8225564
```

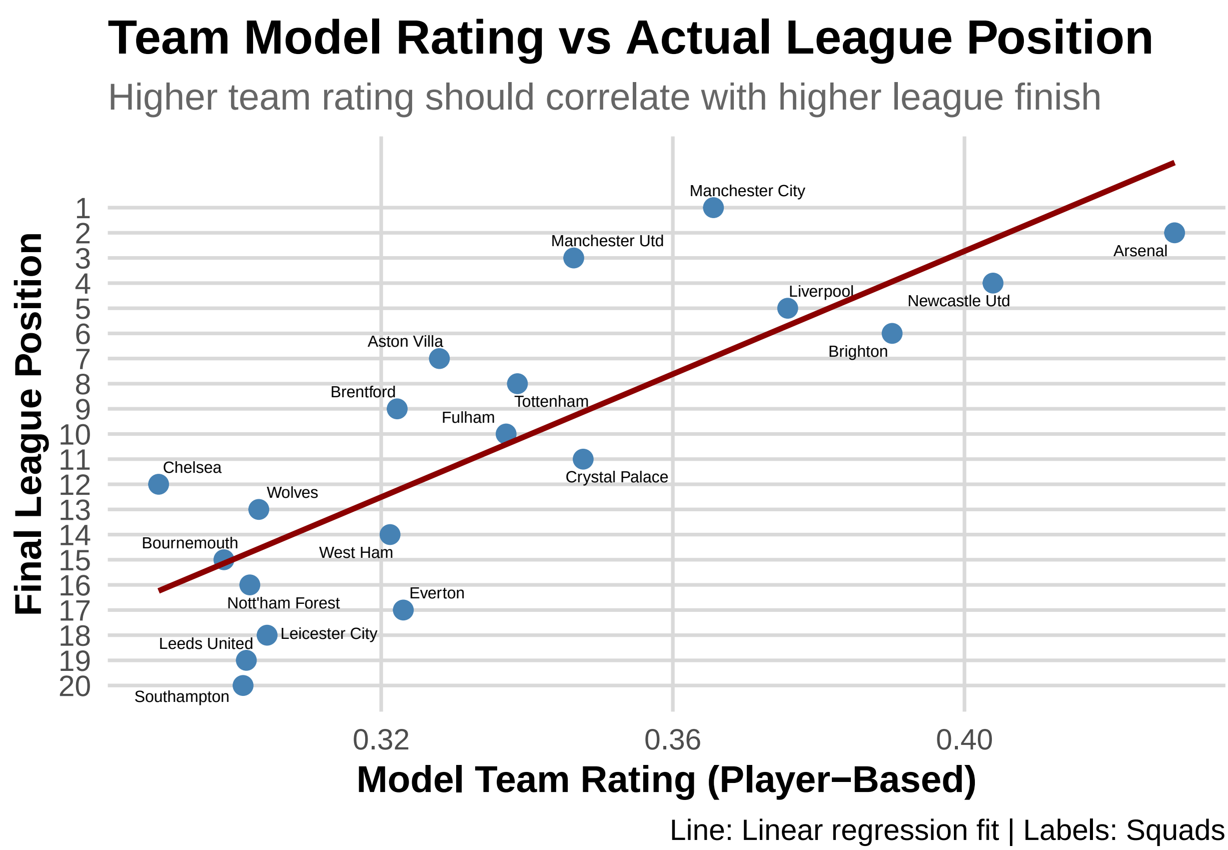
```r
cor(pl_combined_table$Weighted_Team_Rating, pl_combined_table$xGD, method = "spearman")
```

```
## [1] 0.8040617
```

```r
ggplot(pl_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +
  geom_point(size = 3, color = "steelblue") +
  geom_text_repel(size = 2, max.overlaps = 20) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +
  scale_y_reverse(breaks = 1:max(pl_combined_table$Position, na.rm = TRUE)) +
  labs(
    title = "Team Model Rating vs Actual League Position",
    subtitle = "Higher team rating should correlate with higher league finish",
    x = "Model Team Rating (Player-Based)",
    y = "Final League Position",
    caption = "Line: Linear regression fit | Labels: Squads"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    plot.subtitle = element_text(size = 14, color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank()
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a `group` aesthetic or to convert a numerical
##   variable into a factor?
```




```
laliga_ratings_table <- laliga_team_ratings %>% select(Squad, Weighted_Team_Rating)
laliga_table <- laliga_league_table %>% select(Squad) %>% rownames_to_column()
colnames(laliga_table)[1] <- "Position"
laliga_xg_table <- laliga_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

laliga_combined_table <- laliga_ratings_table %>%
  left_join(laliga_table, by = c("Squad")) %>%
  left_join(laliga_xg_table, by = c("Squad"))

laliga_combined_table$Position <- as.numeric(laliga_combined_table$Position)

cor(laliga_combined_table$Weighted_Team_Rating, laliga_combined_table$Position, method = "spearman")
```

La Liga

```
## [1] -0.7052632
```

```
cor(laliga_combined_table$Weighted_Team_Rating, laliga_combined_table$Pts, method = "spearman")
```

```
## [1] 0.6983441
```

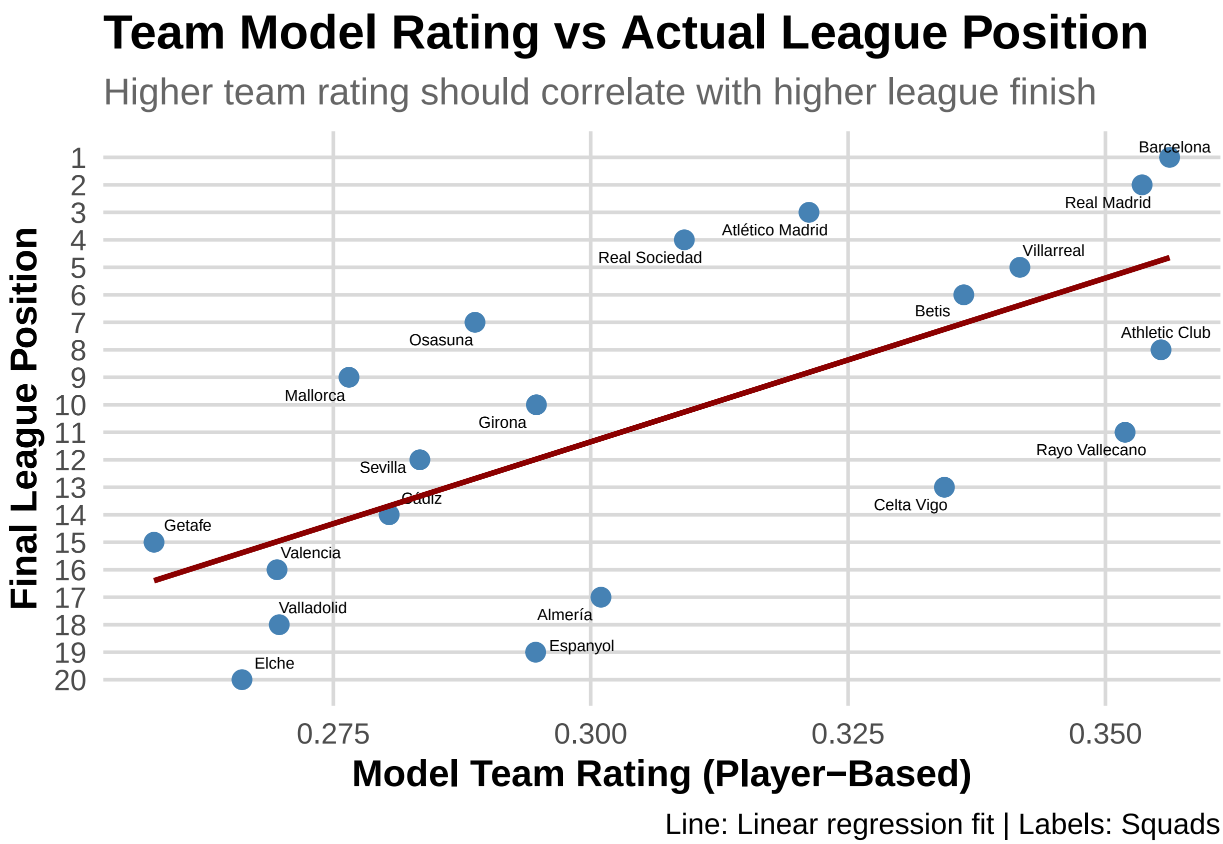
```r
cor(laliga_combined_table$Weighted_Team_Rating, laliga_combined_table$xGD, method = "spearman")
```

```
## [1] 0.7172932
```

```r
ggplot(laliga_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +
  geom_point(size = 3, color = "steelblue") +
  geom_text_repel(size = 2, max.overlaps = 20) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +
  scale_y_reverse(breaks = 1:max(laliga_combined_table$Position, na.rm = TRUE)) +
  labs(
    title = "Team Model Rating vs Actual League Position",
    subtitle = "Higher team rating should correlate with higher league finish",
    x = "Model Team Rating (Player-Based)",
    y = "Final League Position",
    caption = "Line: Linear regression fit | Labels: Squads"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    plot.subtitle = element_text(size = 14, color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank()
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a `group` aesthetic or to convert a numerical
##   variable into a factor?
```



Ligue 1

```r
ligue1_ratings_table <- ligue1_team_ratings %>% select(Squad, Weighted_Team_Rating)
ligue1_table <- ligue1_league_table %>% select(Squad) %>% rownames_to_column()
colnames(ligue1_table)[1] <- "Position"
ligue1_xg_table <- ligue1_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

ligue1_combined_table <- ligue1_ratings_table %>%
  left_join(ligue1_table, by = c("Squad")) %>%
  left_join(ligue1_xg_table, by = c("Squad"))

ligue1_combined_table$Position <- as.numeric(ligue1_combined_table$Position)

cor(ligue1_combined_table$Weighted_Team_Rating, ligue1_combined_table$Position, method = "spearman")
```

```
## [1] -0.8180451
```

```r
cor(ligue1_combined_table$Weighted_Team_Rating, ligue1_combined_table$Pts, method = "spearman")
```

```
## [1] 0.8180451
```

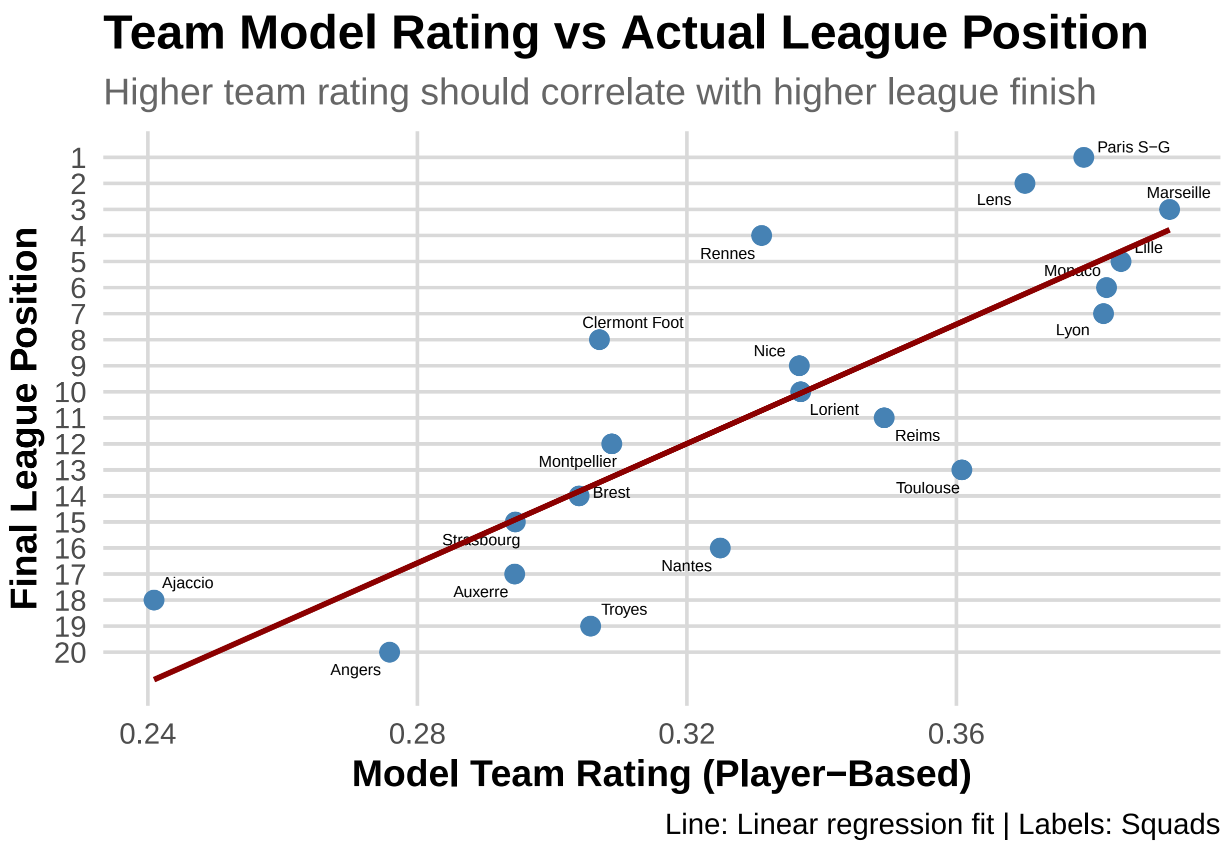
```r
cor(ligue1_combined_table$Weighted_Team_Rating, ligue1_combined_table$xGD, method = "spearman")
```

```
## [1] 0.8115834
```

```r
ggplot(ligue1_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +
  geom_point(size = 3, color = "steelblue") +
  geom_text_repel(size = 2, max.overlaps = 20) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +
  scale_y_reverse(breaks = 1:max(ligue1_combined_table$Position, na.rm = TRUE)) +
  labs(
    title = "Team Model Rating vs Actual League Position",
    subtitle = "Higher team rating should correlate with higher league finish",
    x = "Model Team Rating (Player-Based)",
    y = "Final League Position",
    caption = "Line: Linear regression fit | Labels: Squads"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    plot.subtitle = element_text(size = 14, color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank()
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a `group` aesthetic or to convert a numerical
##   variable into a factor?
```




```
bundesliga_ratings_table <- bundesliga_team_ratings %>% select(Squad, Weighted_Team_Rating)
bundesliga_table <- bundesliga_league_table %>% select(Squad) %>% rownames_to_column()
colnames(bundesliga_table)[1] <- "Position"
bundesliga_xg_table <- bundesliga_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

bundesliga_combined_table <- bundesliga_ratings_table %>%
  left_join(bundesliga_table, by = c("Squad")) %>%
  left_join(bundesliga_xg_table, by = c("Squad"))

bundesliga_combined_table$Position <- as.numeric(bundesliga_combined_table$Position)

cor(bundesliga_combined_table$Weighted_Team_Rating, bundesliga_combined_table$Position, method = "spearm
```

Bundesliga

```
## [1] -0.5562436
```

```
cor(bundesliga_combined_table$Weighted_Team_Rating, bundesliga_combined_table$Pts, method = "spearman")
```

```
## [1] 0.5633082
```

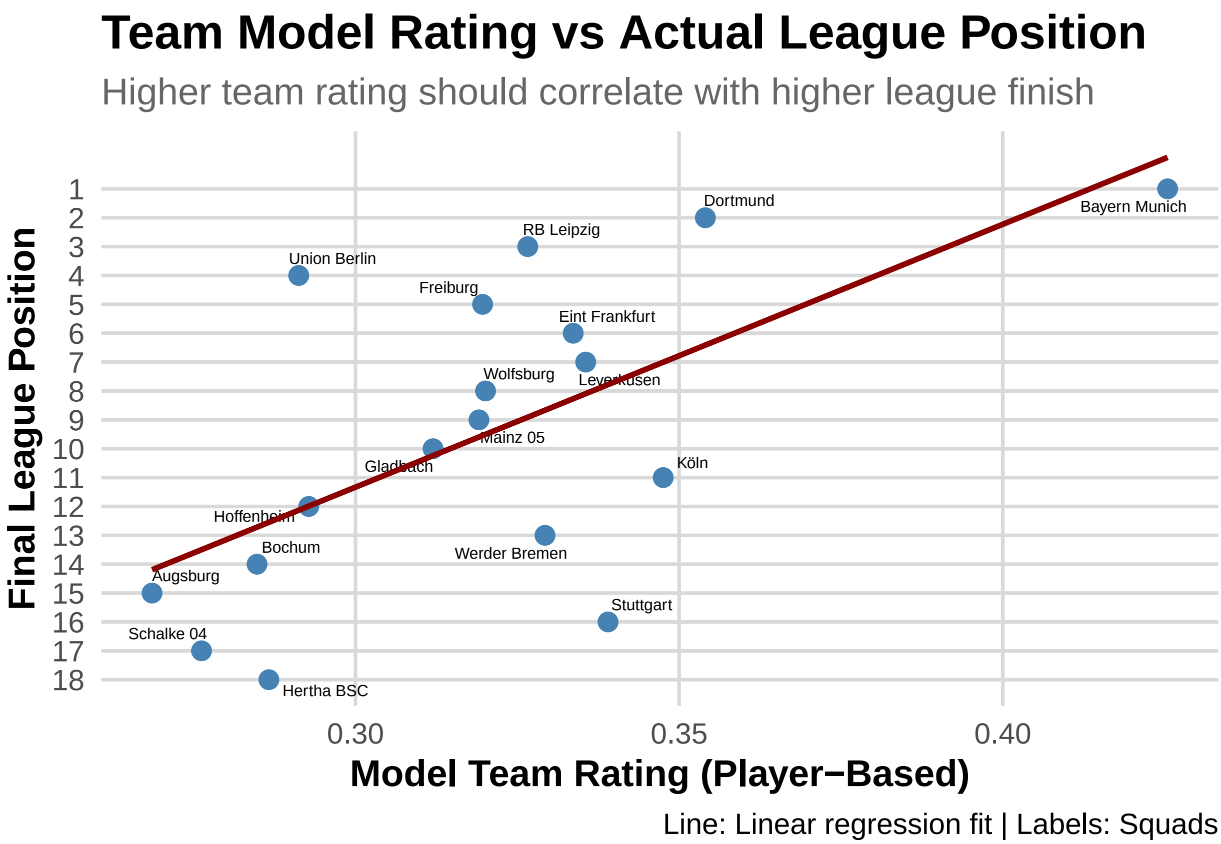
```r
cor(bundesliga_combined_table$Weighted_Team_Rating, bundesliga_combined_table$xGD, method = "spearman")
```

```
## [1] 0.8735159
```

```r
ggplot(bundesliga_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +
  geom_point(size = 3, color = "steelblue") +
  geom_text_repel(size = 2, max.overlaps = 20) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +
  scale_y_reverse(breaks = 1:max(bundesliga_combined_table$Position, na.rm = TRUE)) +
  labs(
    title = "Team Model Rating vs Actual League Position",
    subtitle = "Higher team rating should correlate with higher league finish",
    x = "Model Team Rating (Player-Based)",
    y = "Final League Position",
    caption = "Line: Linear regression fit | Labels: Squads"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    plot.subtitle = element_text(size = 14, color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank()
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a `group` aesthetic or to convert a numerical
##   variable into a factor?
```




```r
seriea_ratings_table <- seriea_team_ratings %>% select(Squad, Weighted_Team_Rating)
seriea_table <- seriea_league_table %>% select(Squad) %>% rownames_to_column()
colnames(seriea_table)[1] <- "Position"
seriea_xg_table <- seriea_league_table %>% select(Squad, Pts, xGD) %>% arrange(desc(xGD))

seriea_combined_table <- seriea_ratings_table %>%
  left_join(seriea_table, by = c("Squad")) %>%
  left_join(seriea_xg_table, by = c("Squad"))

seriea_combined_table$Position <- as.numeric(seriea_combined_table$Position)

cor(seriea_combined_table$Weighted_Team_Rating, seriea_combined_table$Position, method = "spearman")
```

Serie A

```
## [1] -0.7398496
```

```r
cor(seriea_combined_table$Weighted_Team_Rating, seriea_combined_table$Pts, method = "spearman")
```

```
## [1] 0.7423844
```

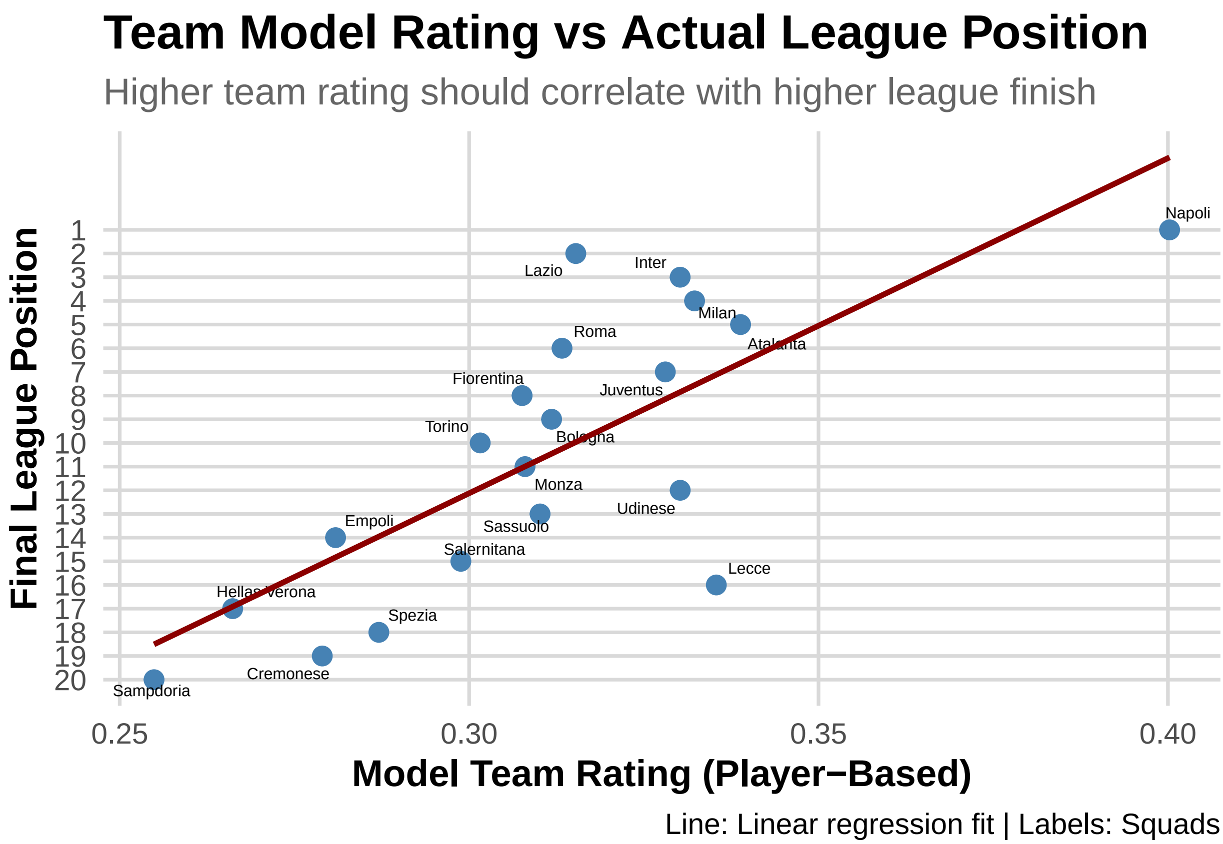
```r
cor(seriea_combined_table$Weighted_Team_Rating, seriea_combined_table$xGD, method = "spearman")
```

```
## [1] 0.7619406
```

```r
ggplot(seriea_combined_table, aes(x = Weighted_Team_Rating, y = Position, label = Squad)) +
  geom_point(size = 3, color = "steelblue") +
  geom_text_repel(size = 2, max.overlaps = 20) +
  geom_smooth(method = "lm", se = FALSE, color = "darkred", linewidth = 1) +
  scale_y_reverse(breaks = 1:max(seriea_combined_table$Position, na.rm = TRUE)) +
  labs(
    title = "Team Model Rating vs Actual League Position",
    subtitle = "Higher team rating should correlate with higher league finish",
    x = "Model Team Rating (Player-Based)",
    y = "Final League Position",
    caption = "Line: Linear regression fit | Labels: Squads"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(face = "bold", size = 18),
    plot.subtitle = element_text(size = 14, color = "gray40"),
    axis.title = element_text(face = "bold"),
    panel.grid.major = element_line(color = "gray85"),
    panel.grid.minor = element_blank()
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: The following aesthetics were dropped during statistical transformation: label.
## i This can happen when ggplot fails to infer the correct grouping structure in
##   the data.
## i Did you forget to specify a `group` aesthetic or to convert a numerical
##   variable into a factor?
```


Team Model Rating vs Actual League Position
Higher team rating should correlate with higher league finish
Final League Position
Model Team Rating (Player−Based)
Line: Linear regression fit | Labels: Squads
0.25
0.30
0.35
0.40
Napoli
Lazio
Inter
Milan
Atalanta
Roma
Juventus
Fiorentina
Torino
Bologna
Monza
Udinese
Sassuolo
Empoli
Salernitana
Lecce
Hellas Verona
Spezia
Cremonese
Sampdoria